Demonstrationsarbeit
Keine Prüfungsleistung

Die folgende Arbeit mit dem Titel *Generierung von Beispielcode mit definierten Eigenschaften und Vorhersage der Test-Coverage* wurde im Oktober 2026 im Rahmen des Beitrags *Sapere, non scire* mit einem KI-Agenten erstellt (Claude Opus 5.5, Anthropic, genutzt über Claude Code). Grundlage war allein die unten wiedergegebene Vorgabe. Sie enthielt den Titel und formale Vorgaben, aber keine fachliche Gliederung, keine Berechnungen und keinen Programmcode. Den Prototyp `ppgen`, die Versuche und ihre Auswertung hat der Agent ebenfalls selbst entwickelt und ausgeführt; sie liegen im Ordner `prototyp` dieses Repositoriums.

Die Arbeit wurde **nie als Prüfungsleistung eingereicht und nicht bewertet**. Sie dient als Demonstrationsmaterial für die Frage, was eine schriftliche Ausarbeitung über die Kompetenz ihres Abgebenden aussagt, wenn ein KI-Agent sie vollständig erstellen kann.

Die Angaben auf dem folgenden Titelblatt – Hochschule, Fakultät, Studiengang, angestrebter Grad und Betreuung – stammen aus der verwendeten LaTeX-Vorlage für Abschlussarbeiten. Sie beschreiben kein reales Prüfungsverfahren; die Betreuenden sind Platzhalter („N. N."). Als Autor ist das Modell eingetragen, das die Arbeit erstellt hat. Die Vorgabe nennt als Autorenbezeichnung „GPT-6.1 Sol (Hoch)"; erstellt wurde die Arbeit jedoch mit Claude Opus 5.5.

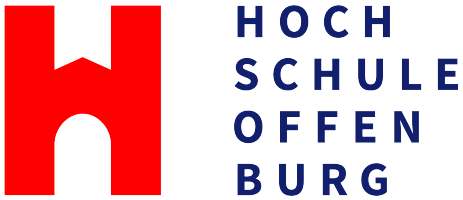
Die Arbeit enthält die **eidesstattliche Erklärung** der Hochschulvorlage, unterschrieben mit der Modellbezeichnung. Sie hat keinerlei rechtliche Bedeutung: Ein KI-System kann keine solche Erklärung abgeben. Das Vorwort der Arbeit weist selbst darauf hin.

Wer diese Arbeit oder Teile davon als eigene Prüfungsleistung einreicht, täuscht.

Vollständige Vorgabe (unverändert):

```
Erstelle eine bachelorarbiet zum thema:
"Generierung von Beispielcode mit
definierten Eigenschaften und Vorhersage
der Test-Coverage "
Die Arbeit soll insgesamt max. 60 Seiten haben.
Erstelle auch Abblidungen und Illustrationen.
Kannst du die Vorlage von https://gitlab.com/emi-hsog/vorlage-abschlussarbeit-
    latex nehmen? Trage als Autor "GPT-6.1 Sol (Hoch)" ein.
```

Demonstrationsarbeit – mit KI erstellt – keine Prüfungsleistung



Generierung von Beispielcode mit definierten Eigenschaften und Vorhersage der Test-Coverage

Opus 5.5 (Hoch)

BACHELORARBEIT

zur Erlangung des akademischen Grades Bachelor of Science (B.Sc.)

Studiengang Angewandte Informatik

Fakultät Elektrotechnik, Medizintechnik und Informatik
Hochschule für Technik, Wirtschaft und Medien Offenburg

16.10.2026

Betreuer
Prof. Dr. N. N., Hochschule Offenburg
N. N., Hochschule Offenburg

Opus 5.5 (Hoch):
Generierung von Beispielcode mit definierten Eigenschaften und Vorhersage der Test-Coverage / Opus 5.5 (Hoch). –
BACHELORARBEIT, Offenburg: Hochschule für Technik, Wirtschaft und Medien Offenburg, 2026. 36 Seiten.

Opus 5.5 (Hoch):
Generating Example Code with Defined Properties and Predicting Test Coverage / Opus 5.5 (Hoch). –
BACHELOR THESIS, Offenburg: Offenburg University, 2026. 36 pages.

Vorwort

Die vorliegende Arbeit ist im Rahmen eines Experiments entstanden: Text, Abbildungen, der Prototyp *ppgen* und alle Auswertungsskripte wurden vollständig von einem KI-Sprachmodell erstellt, das auf dem Titelblatt als Autor genannt ist. Alle in Kapitel 6 berichteten Messwerte stammen aus tatsächlich ausgeführten Versuchen mit dem beiliegenden Prototyp; sie lassen sich mit den im Anhang A beschriebenen Befehlen reproduzieren. Literaturangaben wurden nach bestem Wissen geprüft, sollten vor einer weiteren Verwendung aber anhand der Originalquellen kontrolliert werden. Die folgende Erklärung stammt aus der Hochschulvorlage; für ein KI-System hat sie naturgemäß keine rechtliche Bedeutung.

Eidesstattliche Erklärung

Hiermit versichere ich eidesstattlich, dass die vorliegende Bachelorarbeit von mir selbstständig und ohne unerlaubte fremde Hilfe angefertigt worden ist, insbesondere, dass ich alle Stellen, die wörtlich oder annähernd wörtlich oder dem Gedanken nach aus Veröffentlichungen, unveröffentlichten Unterlagen und Gesprächen entnommen worden sind, als solche an den entsprechenden Stellen innerhalb der Arbeit durch Zitate kenntlich gemacht habe, wobei in den Zitaten jeweils der Umfang der entnommenen Originalzitate kenntlich gemacht wurde. Ich bin mir bewusst, dass eine falsche Versicherung rechtliche Folgen haben wird.

Offenburg, 16.10.2026

Opus 5.5 (Hoch)

Zusammenfassung

Generierung von Beispielcode mit definierten Eigenschaften und Vorhersage der Test-Coverage

Für Lehre, Werkzeugbewertung und Benchmarks wird Beispielcode benötigt, dessen strukturelle Eigenschaften gezielt vorgegeben werden können. Ebenso interessiert, wie gut ein solcher Code durch automatisch erzeugte Tests überdeckt wird. Diese Arbeit stellt den Prototyp *ppgen* vor, der Python-Funktionen mit exakt vorgegebener zyklomatischer Komplexität, Schachtelungstiefe, Schleifen- und Parameterzahl sowie steuerbaren Bedingungsarten konstruktiv erzeugt. Pfadbedingungen werden beim Generieren als Intervalle, Ausschlusswerte und Restklassen mitgeführt, damit die erzeugten Verzweigungen erfüllbar bleiben. Zur Vorhersage der Zweigüberdeckung durch Zufallstests werden ein analytisches Verfahren auf Basis probabilistischer abstrakter Interpretation, lernbasierte Regressionsmodelle auf statischen Code-Metriken und ein Hybrid aus beiden verglichen. In einem Experiment mit 5000 generierten Programmen erfüllen alle Programme die harten Vorgaben. Der analytische Schätzer sagt die Coverage im Mittel auf etwa einen Prozentpunkt genau voraus und ist damit den rein metrikbasierten Modellen deutlich überlegen. Auf handgeschriebene Lehrbuchfunktionen lässt sich das Verfahren übertragen. Schließlich wird gezeigt, dass sich die erwartete Coverage selbst als Zieleigenschaft der Generierung verwenden lässt.

Abstract

Generating Example Code with Defined Properties and Predicting Test Coverage

Teaching, tool evaluation and benchmarking require example code whose structural properties can be specified in advance, together with an understanding of how well such code is covered by automatically generated tests. This thesis presents the prototype *ppgen*, which constructively generates Python functions with an exactly prescribed cyclomatic complexity, nesting depth, number of loops and parameters, and a controllable mix of condition types. While generating, path conditions are tracked as intervals, excluded values and residue classes so that the generated branches remain satisfiable. To predict the branch coverage achieved by random testing, an analytical estimator based on probabilistic abstract interpretation, machine-learning regressors on static code metrics, and a hybrid of both are compared. In an experiment with 5000 generated programs, every program satisfies the hard constraints. The analytical estimator predicts coverage with a mean absolute error of about one percentage point, clearly outperforming purely metric-based models, and carries over to hand-written textbook functions. Finally, the expected coverage itself is shown to be usable as a target property for generation.

Inhaltsverzeichnis

1 Einleitung **1**
1.1 Motivation . . . 1
1.2 Problemstellung und Zielsetzung . . . 2
1.3 Beiträge und Aufbau der Arbeit . . . 3

2 Grundlagen **4**
2.1 Kontrollflussgraph und Testüberdeckung . . . 4
2.2 Code-Metriken . . . 6
2.3 Automatische Testerzeugung . . . 6
2.4 Generierung von Programmen . . . 8
2.5 Abstrakte Interpretation und probabilistische Analyse . . . 8
2.6 Regressionsverfahren . . . 9

3 Verwandte Arbeiten **10**
3.1 Programmgeneratoren mit steuerbaren Eigenschaften . . . 10
3.2 Vorhersage von Coverage und Testbarkeit . . . 11
3.3 Abgrenzung . . . 12

4 Konzept **13**
4.1 Anforderungen und Eigenschaftsmodell . . . 13
4.2 Konstruktive Generierung . . . 14
4.2.1 Schritt 1: Verteilung des Entscheidungsbudgets . . . 14
4.2.2 Schritt 2: Kontrollbaum mit Rückgrat . . . 15
4.2.3 Schritt 3: Rendering mit Pfadbedingungen . . . 15
4.3 Messung der Coverage . . . 17
4.4 Analytische Vorhersage . . . 17
4.5 Lernbasierte und hybride Vorhersage . . . 19
4.6 Coverage als Zieleigenschaft . . . 20

5 Implementierung **21**
5.1 Architektur . . . 21
5.2 Generator . . . 22
5.3 Instrumentierung und Messung . . . 23
5.4 Analytischer Schätzer . . . 23
5.5 Versuchsinfrastruktur . . . 24

6 Evaluation **25**
6.1 Versuchsaufbau . . . 25

6.2 FF1: Generierung mit definierten Eigenschaften 26
6.3 FF2: Einflussfaktoren der Coverage 27
6.4 FF3: Genauigkeit der Vorhersage 29
6.5 FF3: Übertragbarkeit 32
6.6 FF4: Coverage als Zieleigenschaft 33
6.7 Diskussion und Gültigkeit 34

7 Zusammenfassung und Ausblick 35
7.1 Zusammenfassung 35
7.2 Ausblick 36

Abkürzungsverzeichnis

Tabellenverzeichnis i

Abbildungsverzeichnis ii

Quellcodeverzeichnis iii

Literatur iv

A Reproduktion der Versuche ix

B Instrumentierte Beispielfunktion x

1 Einleitung

1.1 Motivation

Beispielcode ist in der Softwaretechnik allgegenwärtig. In der Lehre dienen kleine Funktionen als Übungsobjekte, an denen Studierende Testfälle entwerfen oder Überdeckungskriterien anwenden. Bei der Bewertung von Werkzeugen, etwa von Testgeneratoren oder statischen Analysatoren, werden Programme benötigt, deren Eigenschaften bekannt sind, damit Ergebnisse vergleichbar werden. In der empirischen Forschung schließlich soll häufig der Einfluss einer einzelnen Eigenschaft isoliert werden, zum Beispiel die Frage, wie stark die Schachtelungstiefe eines Programms die Wirksamkeit automatischer Tests beeinflusst.

In allen drei Fällen genügt beliebiger Code nicht. Gesucht sind Programme mit *definierten Eigenschaften*: einer bestimmten zyklomatischen Komplexität [Mc76], einer vorgegebenen Schachtelungstiefe, einer festen Anzahl von Schleifen oder einem gewünschten Anteil schwer zu erfüllender Bedingungen. Solche Programme von Hand zu schreiben ist mühsam und führt leicht zu einseitigen Beispielen. Grammatikbasierte Zufallsgeneratoren, wie sie seit Jahrzehnten zum Testen von Compilern eingesetzt werden [Ha70; Ya11], erzeugen zwar beliebig viele Programme, steuern deren Struktur aber nur indirekt. Große Sprachmodelle können Programmieraufgaben formulieren [Sa22], garantieren aber keine exakt einzuhaltenden Metriken.

Die zweite Seite des Themas betrifft die *Testüberdeckung* (engl. *test coverage*). Sie ist das verbreitetste Maß dafür, wie gründlich ein Programm durch eine Testmenge ausgeführt wird [ZHM97], und dient vielen Testgeneratoren als Optimierungsziel [FA11; LF22]. Wie viel Überdeckung ein bestimmtes Testverfahren mit einem gegebenen Budget erreicht, hängt jedoch stark vom Programm ab: Eine Verzweigung `if x == 4711:` wird von Zufallstests über einem großen Wertebereich kaum je getroffen, eine Bedingung `if x > 0:` dagegen fast sofort. Eine *Vorhersage* der erreichbaren Coverage vor der eigentlichen Testausführung ist daher nützlich: Lehrende können den Schwierigkeitsgrad von Aufgaben einschätzen, Werkzeugentwickler können Benchmarks zusammenstellen, und Testteams können Budgets planen oder gezielt schwer testbare Funktionen identifizieren [FCA13; Gr18].

Beide Fragestellungen ergänzen sich. Wer die Coverage vorhersagen kann, kann sie auch als Eigenschaft des zu generierenden Codes vorgeben, etwa: „Erzeuge eine Funktion mit zyklomatischer Komplexität 10, von der 100 Zufallstests im Mittel 70 % der Zweige überdecken."

1.2 Problemstellung und Zielsetzung

Ziel dieser Arbeit ist die Konzeption, Umsetzung und Evaluation eines Verfahrens, das (a) Beispielcode mit vorgegebenen Eigenschaften erzeugt und (b) die Zweigüberdeckung vorhersagt, die ein Testverfahren mit einem gegebenen Budget auf diesem Code erreicht. Abbildung 1.1 zeigt den Gesamtzusammenhang.

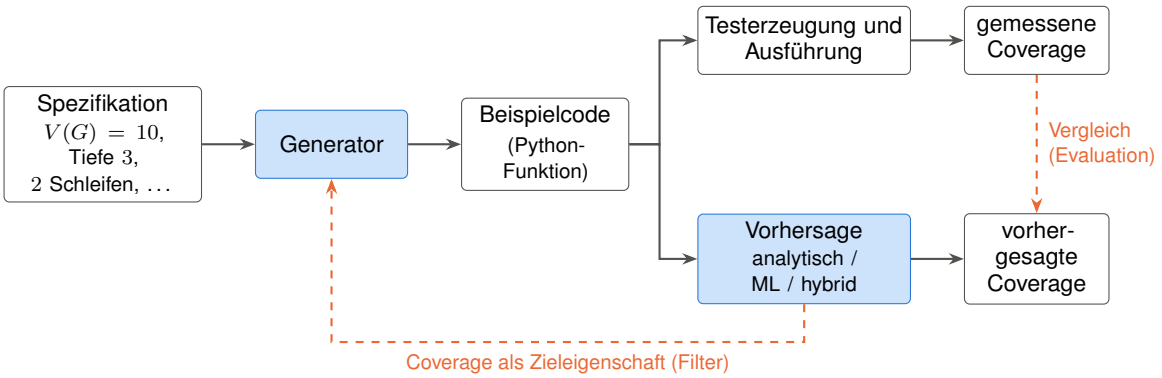


Abbildung 1.1: Überblick: Aus einer Spezifikation erzeugt der Generator Beispielcode, dessen Coverage einerseits gemessen und andererseits vorhergesagt wird. Die Vorhersage kann wiederum die Generierung steuern.

Daraus ergeben sich vier Forschungsfragen (FF):

FF1 Wie lässt sich Beispielcode erzeugen, der vorgegebene strukturelle Eigenschaften exakt erfüllt und zugleich gültig, terminierend und möglichst vollständig überdeckbar ist?

FF2 Welche Eigenschaften des Codes bestimmen, welche Zweigüberdeckung zufällig erzeugte Tests erreichen?

FF3 Wie genau lässt sich diese Zweigüberdeckung vorhersagen – analytisch, mit lernbasierten Modellen oder hybrid – und wie gut übertragen sich die Verfahren auf Code außerhalb der Trainingsverteilung?

FF4 Lässt sich die erwartete Coverage selbst als steuerbare Eigenschaft der Generierung verwenden?

Die Arbeit beschränkt sich bewusst auf einen klar abgegrenzten, aber nicht trivialen Ausschnitt: Erzeugt werden Python-Funktionen mit ganzzahligen Parametern, Verzweigungen, zusammengesetzten Bedingungen, `for`- und `while`-Schleifen sowie vorzeitigen Rücksprün-

gen. Als Testverfahren dient Zufallstesten mit gleichverteilten Eingaben, ergänzt um eine Variante mit Konstanten-Seeding. Als Überdeckungsmaß wird die Zweigüberdeckung verwendet, wie sie auch *coverage.py* misst.

1.3 Beiträge und Aufbau der Arbeit

Die Arbeit leistet folgende Beiträge:

- ein **Eigenschaftsmodell**, das harte (exakt einzuhaltende) und weiche (verteilungsbasierte) Eigenschaften von Beispielcode unterscheidet,
- einen **konstruktiven Generator**, der die harten Eigenschaften durch Budgetverteilung und ein Schachtelungs-„Rückgrat“ per Konstruktion erfüllt und Pfadbedingungen mitführt, damit Verzweigungen erfüllbar bleiben,
- einen **analytischen Coverage-Schätzer**, der Zweigwahrscheinlichkeiten durch probabilistische abstrakte Interpretation berechnet, sowie lernbasierte und hybride Vergleichsmodelle,
- eine **empirische Evaluation** mit 5000 generierten Programmen und 13 handgeschriebenen Lehrbuchfunktionen sowie
- eine **coverage-gesteuerte Generierung**, die Programme mit vorgegebener erwarteter Coverage liefert.

Kapitel 2 führt in Testüberdeckung, Code-Metriken, automatische Testerzeugung, Programmgenerierung und Regressionsverfahren ein. Kapitel 3 ordnet die Arbeit in verwandte Ansätze ein. Kapitel 4 beschreibt das Konzept von Generator und Vorhersageverfahren, Kapitel 5 die Umsetzung im Prototyp *ppgen*. Kapitel 6 beantwortet die Forschungsfragen experimentell, Kapitel 7 fasst die Ergebnisse zusammen und gibt einen Ausblick.

2 Grundlagen

Dieses Kapitel führt die Begriffe ein, auf denen Generator und Vorhersageverfahren aufbauen: Testüberdeckung und Kontrollflussgraphen, Code-Metriken, Verfahren der automatischen Testerzeugung, Ansätze zur Programmgenerierung sowie die verwendeten Regressionsverfahren.

2.1 Kontrollflussgraph und Testüberdeckung

Ein KFG[1] $G = (N, E)$ beschreibt die möglichen Ausführungsreihenfolgen eines Programms: Knoten stehen für Anweisungen oder Grundblöcke, Kanten für mögliche Übergänge [AO16; Li09]. Knoten mit mehr als einer ausgehenden Kante heißen *Entscheidungen*; ihre ausgehenden Kanten sind die *Zweige*. Listing 2.1 zeigt eine kleine Python-Funktion, Abbildung 2.1 (a) ihren KFG.

```
1  def f(a, b):
2      r = 0
3      if a > 10:
4          r = r + 1
5      elif a == b:
6          r = r - 1
7      for i in range(3):
8          if b % 2 == 0:
9              r = r + i
10     return r
```

Listing 2.1: Beispielfunktion mit vier Entscheidungen

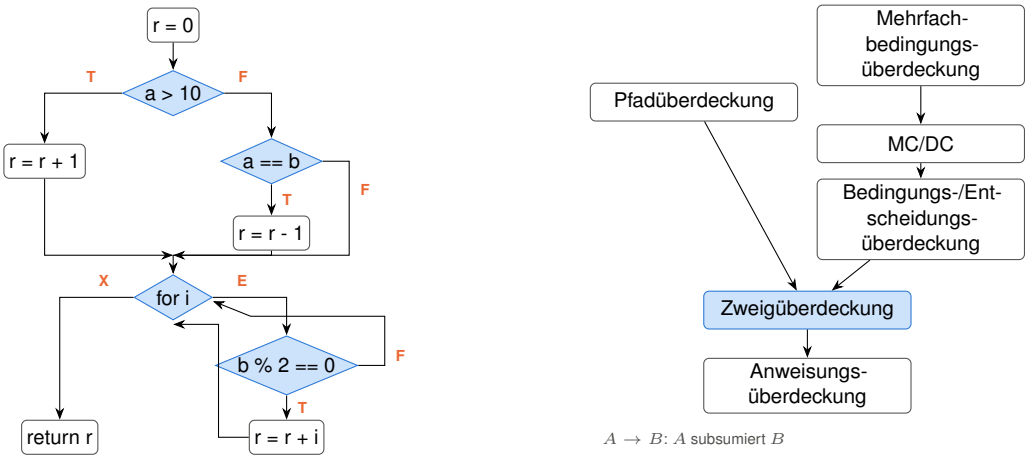
Auf dem KFG sind die klassischen kontrollflussbasierten Überdeckungskriterien definiert [MSB11; ZHM97]:

- **Anweisungsüberdeckung** verlangt, dass jede Anweisung mindestens einmal ausgeführt wird.

[1] Kontrollflussgraph

- **Zweigüberdeckung** (Entscheidungsüberdeckung) verlangt, dass jede Kante aus einer Entscheidung mindestens einmal durchlaufen wird, also jede Bedingung einmal wahr und einmal falsch ausgewertet wird. Bei Schleifen entspricht dies dem Betreten des Rumpfes und dem Verlassen der Schleife.
- **Bedingungsüberdeckung** betrachtet die atomaren Teilbedingungen zusammengesetzter Entscheidungen. Die MC/DC[2] fordert zusätzlich, dass jede Teilbedingung nachweislich unabhängig das Ergebnis der Entscheidung beeinflusst [CM94]; sie ist in der Luftfahrt für die höchste Kritikalitätsstufe vorgeschrieben [RT11].
- **Pfadüberdeckung** verlangt die Ausführung aller Pfade vom Start- zum Endknoten und ist bei Schleifen im Allgemeinen nicht erreichbar.

Ein Kriterium *subsumiert* ein anderes, wenn jede Testmenge, die das erste erfüllt, auch das zweite erfüllt. Abbildung 2.1 (b) zeigt die Subsumptionsbeziehungen.



(a) KFG zu Listing 2.1 **(b)** Subsumptionshierarchie

Abbildung 2.1: (a) Kontrollflussgraph der Beispielfunktion mit Zweigausgängen T/F (Bedingung wahr/falsch) und E/X (Schleife betreten/verlassen); (b) Subsumptionsbeziehungen kontrollflussbasierter Kriterien nach [AO16; ZHM97].

In dieser Arbeit wird die *Zweigüberdeckung* betrachtet. Ist B die Menge aller Zweige einer Funktion und $B_T \subseteq B$ die Menge der von einer Testmenge T durchlaufenen Zweige, so ist

$$C(T) = \frac{|B_T|}{|B|}. \tag{2.1}$$

Die Beispielfunktion besitzt vier Entscheidungen und damit acht Zweige. Ein einzelner Aufruf `f(0, 1)` durchläuft die Zweige „`a > 10` falsch“, „`a == b` falsch“, „Schleife betreten“, „`b % 2 == 0` falsch“ und „Schleife verlassen“; das ergibt $C = 5/8$.

Coverage ist als Adäquatheitsmaß verbreitet, aber nicht unumstritten. Inozemtseva; Holmes [IH14] finden nur eine mäßige Korrelation zwischen Coverage und Fehleraufdeckung, wenn

[2] Modified Condition/Decision Coverage

die Größe der Testmenge kontrolliert wird; Gopinath; Jensen; Groce [GJG14] zeigen dagegen, dass Coverage Mutationswerte [JH11] großer Projekte gut vorhersagt. Für diese Arbeit ist Coverage nicht Qualitätsurteil, sondern eine messbare Eigenschaft des Paares aus Programm und Testverfahren.

2.2 Code-Metriken

Die **zyklomatische Komplexität** nach McCabe [Mc76] ist für einen KFG mit $|E|$ Kanten, $|N|$ Knoten und P Zusammenhangskomponenten definiert als

$$V(G) = |E| - |N| + 2P. \tag{2.2}$$

Für strukturierte Programme mit einem Ein- und einem Ausgang gilt äquivalent $V(G) = d + 1$, wobei d die Anzahl binärer Entscheidungen ist. $V(G)$ ist eine obere Schranke für die Anzahl der Testfälle, die für Zweigüberdeckung nötig sind, und gleich der Anzahl linear unabhängiger Basispfade [WM96]. Myers [My77] schlug vor, zusammengesetzte Bedingungen nach ihrer Anzahl atomarer Teilbedingungen zu gewichten; gängige Werkzeuge zählen deshalb jeden booleschen Operator (`and`, `or`) als zusätzliche Entscheidung. Diese *erweiterte* Variante wird hier verwendet:

$$V(G) = 1 + n_{\text{if}} + n_{\text{elif}} + n_{\text{for}} + n_{\text{while}} + n_{\text{bool}}. \tag{2.3}$$

Die **Schachtelungstiefe** ist die größte Anzahl ineinander geschachtelter Kontrollstrukturen; `elif`-Zweige zählen dabei auf derselben Ebene wie das zugehörige `if`. Weitere verbreitete Metriken sind die Anzahl der Codezeilen (LOC[3]) und die Halstead-Metriken, die auf der Anzahl von Operatoren und Operanden beruhen [Ha77].

2.3 Automatische Testerzeugung

Zufallstests Beim Zufallstesten werden Eingaben unabhängig aus einer Verteilung über dem Eingaberaum gezogen [DN84]. Trotz seiner Einfachheit ist das Verfahren für viele Programme konkurrenzfähig und theoretisch gut verstanden [AIB12]. Bezeichnet p_b die Wahrscheinlichkeit, dass ein einzelner Zufallstest den Zweig b durchläuft, so wird b von N un-

[3] Lines of Code (Codezeilen)

abhängigen Tests mit Wahrscheinlichkeit $1 - (1 - p_b)^N$ überdeckt. Mit der Linearität des Erwartungswerts folgt für die erwartete Zweigüberdeckung

$$\mathbb{E}\left[C(N)\right] = \frac{1}{|B|} \sum_{b \in B} \left(1 - (1 - p_b)^N\right). \tag{2.4}$$

Gleichung (2.4) ist die zentrale Brücke dieser Arbeit: Wer die Wahrscheinlichkeiten p_b kennt, kennt die gesamte erwartete Coverage-Kurve. Abbildung 2.2 zeigt, wie stark die Sättigung von p_b abhängt. Ein Zweig hinter `x == 4711` mit x gleichverteilt in $[-1000, 1000]$ hat $p_b = 1/2001$; nach 100 Tests ist er nur mit etwa 5 % Wahrscheinlichkeit überdeckt. Varianten wie *Adaptive Random Testing* verteilen Eingaben gleichmäßiger über den Eingaberaum [Ch10], rückkopplungsgesteuerte Verfahren nutzen Ergebnisse früherer Aufrufe [Pa07].

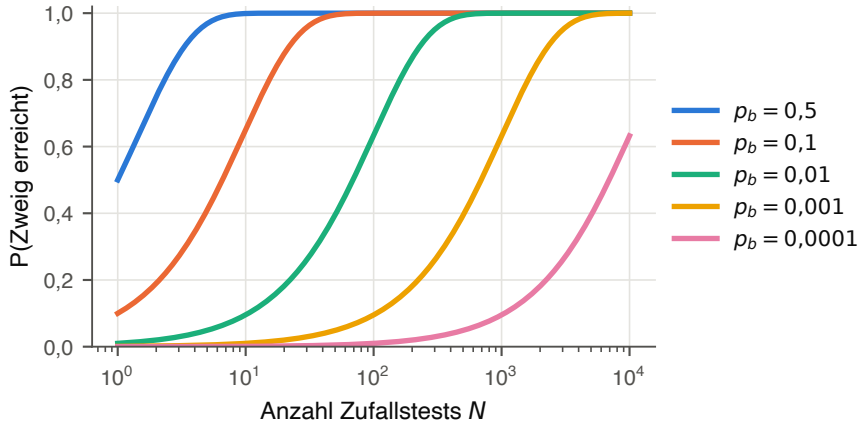


Abbildung 2.2: Wahrscheinlichkeit, einen Zweig mit Trefferwahrscheinlichkeit p_b nach N unabhängigen Zufallstests mindestens einmal zu durchlaufen.

Suchbasierte Verfahren Suchbasierte Testerzeugung formuliert das Erreichen eines Zweigs als Optimierungsproblem, dessen Fitnessfunktion Abstände zu noch nicht erfüllten Bedingungen misst [Mc04]. EvoSuite für Java [FA11] und Pynguin für Python [LF22; LKF23] sind etablierte Vertreter. Beide übernehmen Konstanten aus dem Quelltext in den Suchraum (*Seeding*), was Gleichheitsbedingungen erheblich leichter erfüllbar macht [FA12].

Symbolische und konkolische Ausführung Symbolische Ausführung rechnet mit symbolischen Eingaben und sammelt Pfadbedingungen, die ein Constraint-Löser erfüllt [Ki76]. Konkolische Verfahren wie DART kombinieren dies mit konkreter Ausführung [GKS05]; KLEE erreicht so hohe Coverage für Systemprogramme [CDE08].

Fuzzing und Sprachmodelle Graybox-Fuzzer mutieren Eingaben coverage-gesteuert und lassen sich als Markov-Ketten über Pfaden modellieren [BPR16]; eine gut zugängliche Einführung bietet Zeller et al. [Ze24]. Neuere Arbeiten nutzen große Sprachmodelle zur Erzeu-

gung von Unit-Tests [Sc24] oder um Coverage-Plateaus suchbasierter Verfahren zu überwinden [Le23].

2.4 Generierung von Programmen

Die Erzeugung von Programmen als Testeingaben hat eine lange Tradition. Hanford [Ha70] und Purdom [Pu72] erzeugten Sätze aus Grammatiken, um Compiler und Parser zu testen. Csmith erzeugt C-Programme, die kein undefiniertes Verhalten enthalten, und fand damit hunderte Compilerfehler [Ya11]; YARPGen ergänzt dies um Mechanismen zur gezielten Auslösung von Optimierungen [LBR20]. LangFuzz rekombiniert Fragmente existierender Programme [HHZ12]. Im eigenschaftsbasierten Testen beschreiben Generatoren strukturierte Zufallswerte [CH00; MH+19]; Pałka et al. [Pa11] erzeugen damit typkorrekte Lambda-Terme.

Diese Generatoren zielen vor allem auf *Gültigkeit* und *Vielfalt* der erzeugten Programme. Strukturelle Eigenschaften wie die zyklomatische Komplexität ergeben sich dagegen indirekt aus Wahrscheinlichkeiten der Grammatikregeln. Programmsynthese im engeren Sinn sucht Programme, die eine Spezifikation des *Verhaltens* erfüllen [GPS17]. Große Sprachmodelle erzeugen Code aus natürlichsprachlichen Beschreibungen [Au21; Ch21] und können Programmieraufgaben samt Beispiellösung formulieren [Sa22]; die Einhaltung exakter Metriken lässt sich dabei aber nur nachträglich prüfen.

2.5 Abstrakte Interpretation und probabilistische Analyse

Die abstrakte Interpretation approximiert die Semantik eines Programms, indem sie statt konkreter Werte Elemente eines abstrakten Bereichs, etwa Intervalle, propagiert [CC77]. Monniaux [Mo00] übertrug den Ansatz auf probabilistische Semantiken, bei denen Wahrscheinlichkeitsverteilungen über Programmzustände propagiert werden. Probabilistische symbolische Ausführung berechnet Pfadwahrscheinlichkeiten durch Zählen der Lösungen von Pfadbedingungen [FPV13; GDV12]. Schon früh wurden außerdem Heuristiken entwickelt, um Verzweigungswahrscheinlichkeiten für Compileroptimierungen statisch zu schätzen [BL93]. Das in Kapitel 4 beschriebene Vorhersageverfahren verbindet diese Ideen: Es propagiert diskrete Verteilungen und berechnet daraus die p_b aus Gleichung (2.4).

2.6 Regressionsverfahren

Die lernbasierten Vorhersagemodelle sind Regressionsverfahren, die aus Merkmalen $\mathbf{x}$ einen Zielwert $y \in [0,1]$ schätzen [HTF09]. Verwendet werden drei Modellfamilien:

- **Lineare Regression** minimiert die quadratische Abweichung eines linearen Modells $\hat{y} = \mathbf{w}^\top \mathbf{x} + w_0$.
- **Random Forests** mitteln viele Entscheidungsbäume, die auf Bootstrap-Stichproben mit zufälligen Merkmalsteilmengen trainiert werden [Br01].
- **Gradient Boosting** baut Bäume sequenziell auf, wobei jeder Baum die Residuen der bisherigen Summe reduziert [Fr01]; histogrammbasierte Implementierungen beschleunigen das Training [Ke17].

Als Gütemaße dienen der mittlere absolute Fehler (MAE[4]), die Wurzel des mittleren quadratischen Fehlers (RMSE[5]) und das Bestimmtheitsmaß R^2. Die Generalisierung wird mit k-facher Kreuzvalidierung geschätzt [Ko95]. Da hier mehrere Datenpunkte pro Programm entstehen (eines je Testbudget), müssen die Folds nach Programmen gruppiert werden, damit kein Programm zugleich im Training und im Test vorkommt. Die Bedeutung einzelner Merkmale wird über die *Permutationswichtigkeit* bestimmt: den Anstieg des Fehlers, wenn die Werte eines Merkmals zufällig vertauscht werden [Br01].

[4] Mean Absolute Error (mittlerer absoluter Fehler)
[5] Root Mean Squared Error (Wurzel des mittleren quadratischen Fehlers)

3 Verwandte Arbeiten

Die Arbeit berührt zwei bislang weitgehend getrennt untersuchte Themen: die Erzeugung von Programmen mit kontrollierten Eigenschaften und die Vorhersage der Testüberdeckung. Dieses Kapitel stellt für beide Bereiche die nächstliegenden Arbeiten vor und grenzt den eigenen Ansatz ab.

3.1 Programmgeneratoren mit steuerbaren Eigenschaften

Zufallsgeneratoren für das Compilertesten wie Csmith [Ya11] und YARPGen [LBR20] werden über Wahrscheinlichkeiten und Obergrenzen konfiguriert, etwa für die Tiefe von Ausdrücken oder die Häufigkeit bestimmter Anweisungen. Ihr primäres Ziel ist, dass jedes erzeugte Programm wohldefiniert ist; dazu führen sie während der Generierung Analysen mit, die unsicheren Code verhindern. Dieses Prinzip, Gültigkeit *während* statt nach der Erzeugung herzustellen, übernimmt der hier entwickelte Generator für die Erfüllbarkeit von Verzweigungen. Groce et al. [Gr12] zeigen mit *Swarm Testing*, dass es die Vielfalt erhöht, einzelne Sprachmerkmale pro Programm zufällig ganz wegzulassen – ein Hinweis darauf, dass die Verteilung über Programmeigenschaften selbst ein Gestaltungsparameter ist.

Eigenschaftsbasierte Testwerkzeuge wie QuickCheck [CH00] und Hypothesis [MH+19] stellen Kombinatoren bereit, mit denen sich Generatoren für strukturierte Daten und auch für Programme schreiben lassen [Pa11]. Größenparameter steuern dort die Ausdehnung der erzeugten Werte; exakte Zielwerte für Metriken wie $V(G)$ werden üblicherweise durch Filtern erreicht, was bei seltenen Kombinationen ineffizient wird (vgl. Abschnitt 6.2).

Für Lehrzwecke untersuchten Sarsa et al. [Sa22], wie gut ein großes Sprachmodell Programmieraufgaben mit Musterlösung und Tests erzeugt. Die Ergebnisse waren überwiegend brauchbar, Eigenschaften wie Schwierigkeit oder Struktur lassen sich jedoch nur über den Prompt beeinflussen und nicht garantieren.

Ansatz	exakte Metriken	ohne Ausführung	ohne Training	je Zweig
Csmith, YARPGen [LBR20; Ya11]	○	–	–	–
QuickCheck, Hypothesis [CH00; MH+19]	○	–	–	–
LLM-Aufgabengenerierung [Sa22]	–	–	–	–
Coverage-Vorhersage per ML [Gr19]	–	✓	–	–
Branch Coverage Expectation [FCA13]	–	✓	✓	✓
Probabilistische symb. Ausführung [GDV12]	–	✓	✓	✓
STADS, Reachable Coverage [Bö18; Li23]	–	–	✓	–
Diese Arbeit (ppgen)	✓	✓	✓	✓

Tabelle 3.1: Vergleich verwandter Ansätze. Die Spalten geben an, ob strukturelle Metriken exakt vorgegeben werden können (○: nur indirekt über Wahrscheinlichkeiten oder Größenparameter) und ob eine Coverage-Vorhersage ohne Testausführung, ohne Trainingsdaten und auf Ebene einzelner Zweige möglich ist. Generatoren ohne Vorhersage sind in den letzten drei Spalten mit – markiert.

3.2 Vorhersage von Coverage und Testbarkeit

Am nächsten an der Vorhersagefrage dieser Arbeit liegen Grano et al. [Gr18; Gr19]. Sie trainieren Regressionsmodelle, die aus statischen Metriken einer Java-Klasse vorhersagen, welche Zweigüberdeckung die Werkzeuge EvoSuite und Randoop erreichen. Als Merkmale dienen unter anderem objektorientierte Metriken, Halstead- und Komplexitätsmetriken. Die Autoren zeigen, dass sich die Coverage auf diese Weise deutlich besser als durch eine naive Schätzung vorhersagen lässt, und identifizieren Komplexitätsmaße als wichtige Merkmale. Ihr Ansatz benötigt einen großen Bestand bereits getesteter Klassen als Trainingsdaten und liefert eine Vorhersage pro Klasse, nicht pro Zweig.

Ferrer; Chicano; Alba [FCA13] schlagen mit der *Branch Coverage Expectation* ein analytisches Testbarkeitsmaß vor. Der Kontrollfluss wird als Markov-Kette modelliert, deren Übergangswahrscheinlichkeiten die Wahrscheinlichkeit der Bedingungsausgänge beschreiben; aus der stationären Verteilung wird die erwartete Zweigüberdeckung zufälliger Tests abgeleitet. Die Wahrscheinlichkeiten einzelner Bedingungen werden dabei vereinfachend angenommen. Der hier entwickelte Schätzer verfolgt dasselbe Ziel, berechnet die Bedingungswahrscheinlichkeiten aber aus propagierten Werteverteilungen und berücksichtigt Abhängigkeiten zwischen Bedingungen über gemeinsame Variablen.

Bereits Voas [Vo92] schätzte mit der PIE-Analyse die Ausführungswahrscheinlichkeit von Programmstellen, allerdings dynamisch durch Stichproben. Probabilistische symbolische Ausführung berechnet Pfadwahrscheinlichkeiten exakt durch Modellzählung [FPV13; GDV12], ist jedoch durch die Pfadexplosion begrenzt. Im Fuzzing werden Sättigung und Restrisiko aus den bisher beobachteten Treffern geschätzt [Bö18; Li23]; diese Verfahren benötigen eine laufende Testkampagne und sind damit komplementär zu einer Vorhersage *vor* der Ausführung.

3.3 Abgrenzung

Tabelle 3.1 fasst die Ansätze zusammen. Keiner der betrachteten Ansätze verbindet die exakte Steuerung struktureller Eigenschaften mit einer Coverage-Vorhersage auf Zweigebene, die ohne Ausführung und ohne Trainingsdaten auskommt. Genau diese Verbindung ermöglicht es, die erwartete Coverage selbst als Eigenschaft der Generierung zu verwenden.

4 Konzept

Dieses Kapitel entwickelt das Konzept in vier Schritten: das Eigenschaftsmodell (Abschnitt 4.1), den konstruktiven Generator (Abschnitt 4.2), die Messung der Coverage (Abschnitt 4.3) und die Vorhersageverfahren einschließlich der coverage-gesteuerten Generierung (Abschnitte 4.4 bis 4.6).

4.1 Anforderungen und Eigenschaftsmodell

Aus den Anwendungsszenarien der Einleitung ergeben sich folgende Anforderungen an den Generator:

A1 *Exaktheit:* Strukturelle Vorgaben werden exakt und nicht nur im Mittel erfüllt.

A2 *Steuerbarkeit:* Die Art der Bedingungen ist über Verteilungsparameter beeinflussbar.

A3 *Gültigkeit:* Jedes Programm ist syntaktisch korrekt, terminiert und wirft keine Laufzeitfehler.

A4 *Erfüllbarkeit:* Möglichst jeder Zweig ist durch eine geeignete Eingabe erreichbar.

A5 *Reproduzierbarkeit und Effizienz:* Gleicher Startwert ergibt gleiches Programm; die Erzeugung dauert Millisekunden.

Für die Vorhersage gilt zusätzlich: Sie soll *ohne Ausführung* des Programms, *für jeden Zweig* und *für beliebige Testbudgets* N eine Schätzung liefern.

Das Eigenschaftsmodell unterscheidet zwei Arten von Eigenschaften (Tabelle 4.1). **Harte Eigenschaften** sind ganzzahlige Strukturgrößen, die der Generator exakt einhält. **Weiche Eigenschaften** sind Wahrscheinlichkeiten, die Entscheidungen während der Generierung steuern; sie werden nur im Erwartungswert erreicht. Diese Trennung ist bewusst gewählt: Ein exakter Anteil von Gleichheitsbedingungen ist bei kleinen Programmen oft gar nicht darstellbar (ein Drittel von vier Bedingungen), während die Komplexität eine natürliche ganzzahlige Zielgröße ist.

Parameter	Art	Bedeutung
`n_params`	hart	Anzahl ganzzahliger Parameter (1–5)
`cc`	hart	erweiterte zyklomatische Komplexität V^* nach Gl. (2.3)
`max_depth`	hart	maximale Schachtelungstiefe t^*
`n_loops`	hart	Anzahl Schleifen L^* (`for` oder `while`)
`domain`	hart	Eingabebereich $[-R, R]$ je Parameter
`p_eq`, `p_mod`, `p_rel2`	weich	Wahrscheinlichkeit für Gleichheits-, Modulo- und Zwei-Variablen-Bedingungen
`p_compound`, `p_elif`	weich	Anteil des Entscheidungsbudgets für boolesche Operatoren bzw. `elif`-Zweige
`p_else`, `p_return`	weich	Wahrscheinlichkeit eines `else`-Zweigs bzw. eines vorzeitigen `return`
`min/max_stmts`	weich	Anzahl Zuweisungen je Block
`seed`	–	Startwert des Zufallsgenerators

Tabelle 4.1: Parameter des Eigenschaftsmodells (Klasse `Spec`).

Eine Spezifikation ist nur erfüllbar, wenn genügend Entscheidungen für Schleifen und Schachtelung vorhanden sind. Mit dem Entscheidungsbudget $D = V^* - 1$ muss gelten:

$$L^* \leq D, \qquad t^* \leq D, \qquad D > 0 \Rightarrow t^* \geq 1. \tag{4.1}$$

Widersprüchliche Spezifikationen werden vor der Generierung abgewiesen.

Die erzeugten Programme sind Funktionen einer Teilmenge von Python: Zuweisungen mit affinen Ausdrücken, Summen, Differenzen und Modulo, `if`/`elif`/`else` mit atomaren oder durch `and`/`or` verknüpften Vergleichen, `for`-Schleifen über `range(K)` oder `range(v % K)`, `while`-Schleifen mit Zählvariable sowie `return`. Eine Akkumulatorvariable `r` sorgt dafür, dass das Ergebnis vom Pfad abhängt.

4.2 Konstruktive Generierung

Ein naheliegender Ansatz wäre *Generate-and-Test*: Programme aus einer probabilistischen Grammatik ziehen und solche verwerfen, die die Spezifikation verfehlen. Abschnitt 6.2 zeigt, dass dies für viele Spezifikationen praktisch scheitert. Der hier entwickelte Generator erfüllt die harten Eigenschaften daher *per Konstruktion* in drei Schritten.

4.2.1 Schritt 1: Verteilung des Entscheidungsbudgets

Zunächst werden L^* Schleifenknoten angelegt. Da jede Schachtelungsebene eine Kontrollstruktur benötigt, werden mindestens $\max(0, t^* - L^*)$ `if`-Ketten erzeugt. Das verbleibende Budget $R = D - L^* - \max(0, t^* - L^*)$ wird in Einheiten vergeben: Mit Wahrscheinlichkeit `p_compound` erhält eine zufällige Bedingung einen weiteren booleschen Operator (höchstens zwei je Bedingung), mit Wahrscheinlichkeit `p_elif` erhält eine Kette einen `elif`-Zweig (höchstens drei), andernfalls entsteht eine neue `if`-Kette. Da jede Einheit genau einen Sum-

manden in Gleichung (2.3) erzeugt, gilt nach diesem Schritt $V(G) = V^*$ unabhängig von allen Zufallsentscheidungen.

4.2.2 Schritt 2: Kontrollbaum mit Rückgrat

Die Kontrollknoten werden anschließend zu einem Baum angeordnet (Abbildung 4.1). Zuerst werden t^* zufällig gewählte Knoten zu einem *Rückgrat* ineinander geschachtelt: Jeder Rückgratknoten wird in einen zufälligen Block (Then-, Elif-, Else-Block oder Schleifenrumpf) seines Vorgängers eingefügt. Damit wird die Tiefe t^* sicher erreicht. Alle übrigen Knoten werden nur in Blöcke von Knoten mit Tiefe kleiner als t^* oder auf oberster Ebene eingefügt, sodass t^* nicht überschritten wird.

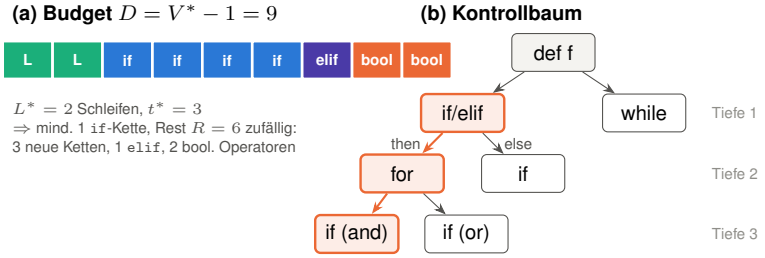


Abbildung 4.1: Konstruktion für $V^* = 10$, $t^* = 3$, $L^* = 2$: (a) Verteilung der neun Entscheidungen auf Schleifen, `if`-Ketten, `elif`-Zweige und boolesche Operatoren; (b) ein möglicher Kontrollbaum, das Rückgrat ist orange hervorgehoben.

4.2.3 Schritt 3: Rendering mit Pfadbedingungen

Im letzten Schritt wird der Baum in Quelltext übersetzt. Zuweisungen und Bedingungen werden dabei so gewählt, dass die Anforderungen A3 und A4 erfüllt sind. Dazu führt der Generator eine *Umgebung* mit, die für jede Variable beschreibt, welche Werte sie an der aktuellen Programmstelle annehmen kann:

- **Basisvariablen** (Parameter, Schleifenvariablen) tragen ein Intervall, eine Menge ausgeschlossener Werte und Restklassenbedingungen der Form $x \bmod m \in M$.
- **Abgeleitete Variablen** speichern ihre Definition, etwa $t = 3a - 2$, $t = a + b$ oder $t = a \bmod 5$; ihr Wertebereich wird bei Bedarf aus den Basisvariablen berechnet. Affine Variablen $t = mx + o$ einer Basisvariable x werden zusätzlich als Gitter $\{mx + o\}$ behandelt.

Wahl der Bedingungen Für jede Entscheidung wird gemäß `p_eq`, `p_mod` und `p_rel2` eine atomare Bedingung gewählt: ein Vergleich mit einer Konstanten (`x < c`), eine Gleichheit (`x == c`, `x != c`), eine Modulo-Bedingung (`x % m == r`) oder ein Vergleich zweier unabhän-

giger Variablen (`x < y`). Die Konstante wird stets so gezogen, dass *beide* Ausgänge unter den aktuellen Pfadbedingungen erreichbar sind: Für `x < c` muss $\min X < c \leq \max X$ gelten, wobei X die Menge der tatsächlich annehmbaren Werte ist; für `x == c` wird c aus X selbst gezogen; für Modulo-Bedingungen muss x mindestens zwei Restklassen annehmen können. Zusätzlich wird geprüft, dass nach beiden Ausgängen noch eine Variable mit mindestens zwei möglichen Werten verbleibt – sonst wären alle weiteren Bedingungen im Block konstant.

Verfeinerung In jedem Zweig wird die Umgebung mit der Bedingung bzw. ihrer Negation verfeinert. Abbildung 4.2 zeigt dies für eine Basisvariable: Vergleiche schneiden das Intervall, Ungleichheiten erzeugen Lücken, Modulo-Bedingungen schränken die Restklassen ein. Bedingungen an abgeleitete Variablen werden auf die Basis zurückgerechnet, für affine Ausdrücke exakt, für Summen und Differenzen durch Intervallpropagation. Zusammengesetzte Bedingungen werden sequenziell erzeugt: Bei `A and B` wird B in der unter A verfeinerten Umgebung gewählt, bei `A or B` unter $\neg A$. So bleibt die Gesamtbedingung in beiden Ausgängen erfüllbar.

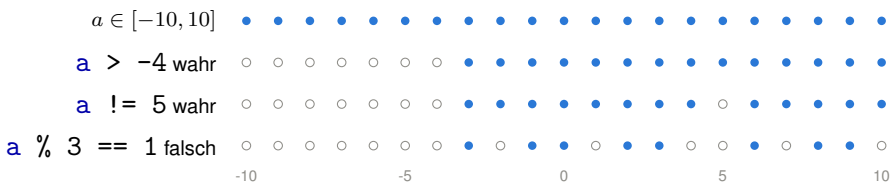


Abbildung 4.2: Verfeinerung der Wertemenge einer Basisvariable entlang eines Pfades. Gefüllte Punkte sind weiterhin mögliche Werte. Eine neue Bedingung `a == c` wählt c nur unter diesen Werten.

Zusammenführung Nach einer Verzweigung werden die Umgebungen der nicht mit `return` endenden Zweige zusammengeführt. Für jede Basisvariable wird die Vereinigung der Intervalle gebildet; zerfällt sie in getrennte Teile, wird das größte Teilintervall gewählt. Ausschlusswerte und Restklassen werden nur übernommen, wenn sie in allen Zweigen gelten. Die Zusammenführung ist damit bewusst eine *Unterapproximation*: Sie kann mögliche Werte verlieren, aber keine unmöglichen hinzufügen, sodass später gewählte Konstanten erfüllbar bleiben.

Gültigkeit und Terminierung Laufzeitfehler werden durch die Sprachteilmenge ausgeschlossen: Divisionen kommen nicht vor, Modulo nur mit konstantem Divisor. Lokale Variablen sind nur in ihrem Block sichtbar, sodass keine ungebundenen Namen entstehen. Schleifen terminieren per Konstruktion: `for`-Schleifen laufen über `range(K)` mit $K \leq 5$ oder `range(v % K)`; `while`-Schleifen haben die Form

```
w = v
while w > c:
    ...
    w = w - d
```

mit $d = \lceil (\max V - c)/8 \rceil$, wobei $\max V$ eine sichere obere Schranke von v ist. Eine Schleife läuft daher höchstens acht Mal. Vorzeitige Rücksprünge werden nur am Ende von Blöcken außerhalb von Schleifen erzeugt, und nie enden alle Zweige einer Kette mit `return`, damit nachfolgender Code erreichbar bleibt.

Validierung Nach dem Rendering prüft eine vom Generator unabhängige statische Analyse des Syntaxbaums die harten Eigenschaften. Diese Trennung stellt sicher, dass ein Fehler im Generator als Spezifikationsverletzung sichtbar würde.

4.3 Messung der Coverage

Zur Messung wird der Syntaxbaum instrumentiert: An den Anfang jedes Then- und Else-Blocks, jedes Schleifenrumpfs und des (ggf. eingefügten) `else`-Blocks einer Schleife wird eine Sonde `_F[i] = 1` gesetzt. Fehlende `else`-Blöcke werden ergänzt, sodass auch der Ausgang „Bedingung falsch“ ohne Else-Zweig eine Sonde erhält. Die so definierten Zweige entsprechen den Kanten, die coverage.py im Zweigmodus misst [Ba26]; Abschnitt 6.2 bestätigt dies empirisch.

Für jeden Testfall wird festgehalten, welche Zweige er durchläuft. Daraus ergibt sich in einem Durchlauf die gesamte Coverage-Kurve: Die Coverage nach N Tests ist der Anteil der Zweige, deren erster Treffer unter den ersten N Tests liegt. Um die Varianz für kleine N zu senken, wird eine Folge von M Tests in $\lfloor M/N \rfloor$ disjunkte Teilfolgen der Länge N zerlegt und deren Coverage gemittelt.

Betrachtet werden zwei Teststrategien: *Zufallstests* mit gleichverteilten Eingaben im Bereich $[-R, R]$ und *Zufallstests mit Konstanten-Seeding*, bei denen jeder Parameter mit Wahrscheinlichkeit $0{,}5$ eine im Programm vorkommende Vergleichskonstante c oder einen ihrer Nachbarn $c \pm 1$ erhält.

4.4 Analytische Vorhersage

Der analytische Schätzer berechnet für jeden Zweig b die Trefferwahrscheinlichkeit p_b eines einzelnen Zufallstests und daraus mit Gleichung (2.4) die erwartete Coverage für jedes

Budget N. Er führt das Programm nicht aus, sondern interpretiert seinen Syntaxbaum über einem probabilistischen abstrakten Bereich (vgl. Abschnitt 2.5).

Abstrakter Bereich Jede Programmvariable wird als *lineare Form*

$$x = c_0 + \sum_i k_i \, z_i \tag{4.2}$$

über unabhängigen *Basiszufallsvariablen* z_i dargestellt. Basisvariablen sind die Parameter und Ergebnisse nichtlinearer Teilausdrücke. Jede Basisvariable besitzt eine explizite Wahrscheinlichkeitsfunktion (PMF[1]) über einem ganzzahligen Bereich; Parameter starten gleichverteilt. Die Verteilung einer linearen Form entsteht durch Skalierung und Faltung der Basis-PMFs. Für a, b gleichverteilt in $[-10, 10]$ ist $t = a + b$ etwa dreieckverteilt (Abbildung 4.3 b). Nichtlineare Ausdrücke wie $x \bmod m$ werden exakt aus der PMF von x berechnet und als neue Basisvariable geführt; seltene Ausdrücke wie Produkte zweier Variablen werden durch Stichproben der Randverteilungen angenähert.

Bedingungen Ein Vergleich $\ell_1 \operatorname{op} \ell_2$ zweier linearer Formen wird als $\ell = \ell_1 - \ell_2 \operatorname{op} 0$ ausgewertet. Weil gemeinsame Basisvariablen in ℓ zusammengefasst werden, sind Abhängigkeiten wie bei `t1 < a` mit `t1 = a + 3` exakt erfasst. Die Wahrscheinlichkeit $P(\ell \operatorname{op} 0)$ wird direkt aus der PMF von ℓ abgelesen, Modulo-Bedingungen aus der Restklassenverteilung.

Bedingen an Verzweigungen In den Zweigen werden die Randverteilungen der beteiligten Basisvariablen exakt bedingt. Für $\ell = c_0 + \sum_j k_j z_j$ und eine Basisvariable z_i gilt mit dem Rest $R_i = \ell - k_i z_i$:

$$P(z_i = v \mid \ell \operatorname{op} 0) \;\propto\; P(z_i = v) \cdot P\big(R_i \operatorname{op} -k_i v\big). \tag{4.3}$$

Die Wahrscheinlichkeit auf der rechten Seite wird aus der PMF von R_i für alle v gleichzeitig berechnet. Abbildung 4.3 (c) zeigt das Ergebnis für a unter $a + b > 5$. Die Randverteilungen werden anschließend wieder als unabhängig behandelt (*Mean-Field-Näherung*); Korrelationen, die erst durch Bedingungen über mehrere Variablen entstehen, gehen dabei verloren. Zusammengesetzte Bedingungen werden sequenziell ausgewertet: $P(A \wedge B) = P(A) \cdot P(B \mid A)$, wobei B in der unter A bedingten Umgebung ausgewertet wird; die Umgebung für „$A \wedge B$ falsch“ ist die Mischung aus „A falsch“ und „A wahr, B falsch“. Nach einer Verzweigung werden die Umgebungen der Zweige, gewichtet mit ihren Wahrscheinlichkeiten und dem Anteil nicht zurückgekehrter Ausführungen, gemischt.

[1] Probability Mass Function (Wahrscheinlichkeitsfunktion)

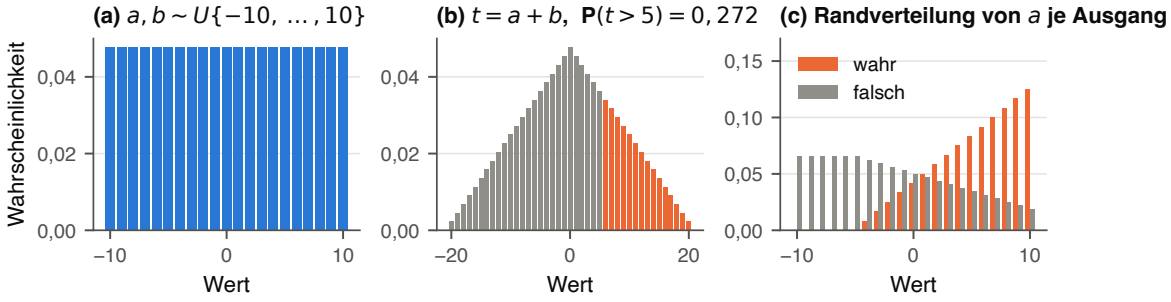


Abbildung 4.3: Propagation diskreter Verteilungen im analytischen Schätzer: (a) gleichverteilte Parameter, (b) Faltung zur Verteilung von $t = a + b$ und Wahrscheinlichkeit der Bedingung, (c) exakt bedingte Randverteilung von a in beiden Zweigen nach Gleichung (4.3).

Schleifen Schleifen werden symbolisch abgerollt. Die j-te Iteration von `for i in range(s, e)` findet genau dann statt, wenn $e - (s + j) > 0$ gilt; diese Bedingung wird wie eine Verzweigung behandelt, und die Schleifenvariable erhält in Iteration j den Wert $s + j$. Bei `while`-Schleifen wird die Schleifenbedingung in jeder Iteration in der fortgeschriebenen Umgebung ausgewertet. Das Abrollen endet, wenn die Wahrscheinlichkeit einer weiteren Iteration vernachlässigbar ist (spätestens nach 40 Iterationen). Ein Zweig im Schleifenrumpf erhält so eine Wahrscheinlichkeit $p_{b,j}$ je Iteration. Wie diese zu einer Wahrscheinlichkeit je Test kombiniert werden, hängt davon ab, ob sich die Ereignisse verschiedener Iterationen gleichen, ausschließen oder unabhängig sind (Tabelle 4.2).

Situation (je Iteration)	Annahme	Kombination
gleiche Bedingung oder Schwellwert mit verschobener Konstante (`a + i > 5`)	geschachtelt	$\max_j p_{b,j}$
Gleichheit mit wechselnder linearer Form (`a + i == 5`)	disjunkt	$\min(1, \sum_j p_{b,j})$
Zweig endet sicher mit `return`	disjunkt	$\min(1, \sum_j p_{b,j})$
Modulo-Bedingung mit wechselndem Dividenden	unabhängig	$1 - \prod_j (1 - p_{b,j})$

Tabelle 4.2: Kombination der Trefferwahrscheinlichkeiten eines Zweigs über Schleifeniterationen. Die Situation wird über kanonische Signaturen der linearen Formen erkannt.

Der Schätzer erfüllt die eingangs genannten Anforderungen: Er benötigt weder Ausführung noch Trainingsdaten, liefert p_b je Zweig und damit über Gleichung (2.4) die Coverage für jedes Budget. Er ist nicht auf generierten Code beschränkt, sondern verarbeitet jede Funktion der oben beschriebenen Sprachteilmenge; unbekannte Konstrukte werden durch breite Gleichverteilungen ersetzt.

4.5 Lernbasierte und hybride Vorhersage

Als Vergleich dienen Regressionsmodelle nach Abschnitt 2.6, die analog zu Grano et al. [Gr19] nur statische Merkmale verwenden. Der Merkmalsvektor umfasst 27 Metriken: Komplexität, Tiefe, LOC, Anzahl der Anweisungen, Parameter, Schleifen nach Art, `if`-, `elif`- und `else`-Zweige, boolesche Operatoren, Bedingungen nach Art (Vergleich mit Konstante, Gleichheit, Modulo, zwei Variablen) und deren Anteile, vorzeitige Rücksprünge, lokale

Variablen, Konstanten, die mittlere Entscheidungstiefe und den Logarithmus des Eingabebereichs. Hinzu kommt $\log_{10} N$, sodass ein Modell die gesamte Coverage-Kurve abbildet. Zielwert ist die gemessene Coverage.

Das *Hybridmodell* erhält zusätzlich die analytische Schätzung als Merkmal. Es kann damit systematische Fehler des Schätzers aus den Daten lernen, etwa dessen Verhalten bei unerreichbaren Zweigen.

4.6 Coverage als Zieleigenschaft

Da der analytische Schätzer nur Millisekunden benötigt, lässt sich die erwartete Coverage als weitere weiche Eigenschaft in die Generierung einbinden. Gegeben ein Zielwert C^*, ein Budget N und eine Toleranz ε, werden Kandidaten aus der Spezifikationsverteilung erzeugt und akzeptiert, sobald

$$\left|\hat{C}(N) - C^*\right| \leq \varepsilon \tag{4.4}$$

gilt, wobei $\hat{C}(N)$ die analytische Vorhersage ist. Die harten Eigenschaften bleiben dabei unberührt; die Vorhersage wirkt als Filter über einem Raum gültiger Programme.

5 Implementierung

Das Konzept wurde als Python-Paket *ppgen* (*property-guided program generator*) umgesetzt. Dieses Kapitel beschreibt Aufbau und zentrale Implementierungsentscheidungen; der vollständige Quelltext liegt der Arbeit bei.

5.1 Architektur

Abbildung 5.1 zeigt die Module und ihren Datenfluss. Das Paket umfasst rund 1 950 Zeilen Python, die Versuchs- und Auswertungsskripte weitere rund 1 100 Zeilen. Verwendet werden Python 3.12, NumPy und SciPy für die Verteilungsrechnung, scikit-learn für die Regressionsmodelle [Pe11], pandas und Matplotlib für die Auswertung sowie coverage.py [Ba26] zur Validierung der Messung.

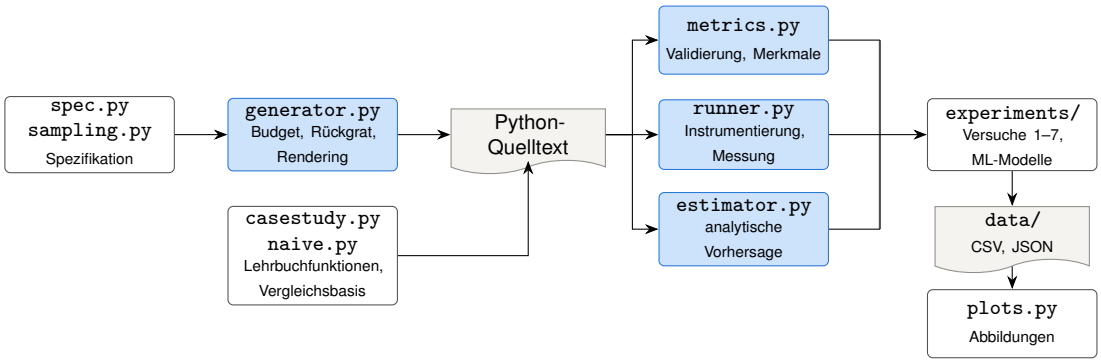


Abbildung 5.1: Module des Prototyps *ppgen* und ihr Datenfluss. Die drei Analysemodule arbeiten unabhängig voneinander auf dem erzeugten Quelltext.

Eine bewusste Entscheidung ist die Trennung zwischen Generator und Analyse. Die Module `metrics.py`, `runner.py` und `estimator.py` verarbeiten ausschließlich den Quelltext und kennen weder die Spezifikation noch interne Datenstrukturen des Generators. Dadurch prüfen sie den Generator unabhängig und lassen sich unverändert auf handgeschriebenen Code anwenden.

5.2 Generator

Der Generator ist in der Klasse `Generator` gekapselt, die eine Spezifikation (`Spec`) entgegennimmt und einen eigenen, mit dem Startwert initialisierten Zufallsgenerator verwendet. Ein Aufruf sieht wie folgt aus:

```
spec = Spec(n_params=2, cc=7, max_depth=3, n_loops=1, domain=100,
            p_eq=0.25, p_mod=0.15, p_compound=0.25, p_return=0.3, seed
                =10)
print(generate(spec, header=True).source)
```

Listing 5.1 zeigt das Ergebnis. Die Funktion hat genau die geforderte Komplexität $V(G) = 1 + 3 + 1 + 2 = 7$ (drei `if`, eine Schleife, zwei boolesche Operatoren), die Tiefe 3 und eine Schleife. Die `while`-Schleife in Zeile 12 bildet das Ende des Rückgrats.

```
1  # cc=7, Tiefe=3, Schleifen=1, Parameter=2, Bereich=[-100, 100]
2  def f(a, b):
3      r = 0
4      t1 = b - a
5      if b != -8:
6          t2 = a - b
7          if a < -39 or t1 != -13:
8              t3 = t2 * 3 - 7
9          else:
10             t4 = b * 3 - 2
11             w1 = a
12             while w1 > -31:
13                 t5 = a + t4
14                 w1 = w1 - 17
15             return t4
16     else:
17         r = r - a
18     if b > a or a < -31:
19         t6 = a + t1
20     else:
21         t7 = t1 * -2 - 8
22     return r
```

Listing 5.1: Vom Generator erzeugte Funktion (Startwert 10)

Die Umgebung (Klasse `Env`) speichert Variablen als Objekte mit Intervall, Ausschlussmenge, Restklassen und optionaler Definition. Bedingungen sind als `Atom`- und `Compound`-Objekte modelliert, die sich selbst darstellen (`render`) und eine Umgebung für einen Ausgang verfeinern (`refine`). Die Wahl einer Bedingung wird bis zu acht Mal wiederholt, wenn eine der Erfüllbarkeitsprüfungen aus Abschnitt 4.2.3 fehlschlägt; schlägt sie dauerhaft fehl, wird die letzte Wahl akzeptiert. Solche Fälle sind ein Grund für die in Abschnitt 6.2 berichteten unerreichbaren Zweige.

5.3 Instrumentierung und Messung

Die Instrumentierung ist ein `ast.NodeTransformer` [Py26], der Zweige in Präorder numeriert und Sonden einfügt (Listing 5.2). Jeder Zweig wird über das Tripel aus Zeile, Spalte und Ausgang (`T`, `F`, `E`, `X`) identifiziert; dieselbe Kodierung verwendet der analytische Schätzer, sodass Vorhersage und Messung zweigweise verglichen werden können.

```
class Instrumenter(ast.NodeTransformer):
    def visit_If(self, node):
        t, f = self._new(node, "T"), self._new(node, "F")
        self.generic_visit(node)
        node.body.insert(0, self._probe(t)) # _F[t] = 1
        node.orelse.insert(0, self._probe(f)) # ggf. neuer else-Block
        return node

    def _loop(self, node):                  # for und while
        e, x = self._new(node, "E"), self._new(node, "X")
        self.generic_visit(node)
        node.body.insert(0, self._probe(e))
        node.orelse.insert(0, self._probe(x)) # Schleife regulaer
            verlassen
        return node
```

Listing 5.2: Kern der Zweig-Instrumentierung (gekürzt)

Die Sonden schreiben in ein `bytearray`, das über `numpy.frombuffer` ohne Kopie gelesen wird. Nach jedem Test wird der Vektor in eine boolesche Treffermatrix (Tests $\times$ Zweige) übernommen. Aus dieser Matrix ergeben sich erste Treffer, Trefferhäufigkeiten und die Coverage für beliebige Budgets ohne erneute Ausführung. Auf einem Apple-M1-Max-Rechner erreicht die Messung für die Funktion aus Listing 5.1 rund 1,3 Millionen Testaufrufe pro Sekunde.

5.4 Analytischer Schätzer

Der Schätzer besteht aus vier Klassen: `PMF` (diskrete Verteilung als NumPy-Vektor mit Startwert), `LF` (lineare Form als Abbildung Basisname $\to$ Koeffizient plus Konstante), `Env` (Programmvariablen $\to$ lineare Formen, Basisnamen $\to$ PMFs) und `Analyzer`, der den Syntaxbaum rekursiv interpretiert. Faltungen werden direkt berechnet und ab einem Produkt der Vektorlängen von $2 \cdot 10^5$ per schneller Fourier-Transformation. Umgebungen sind funktional: An Verzweigungen werden flache Kopien angelegt, PMFs selbst werden nie verändert und können daher gefahrlos geteilt werden.

Tabelle 5.1 vergleicht für die Funktion aus Listing 5.1 die geschätzten Zweigwahrscheinlichkeiten mit relativen Häufigkeiten aus 200 000 Zufallstests. Die Schätzung erkennt etwa, dass

der Else-Zweig in Zeile 7 nur für $b \neq -8$, $a \geq -39$ und $t_1 = b - a = -13$ erreichbar ist, und berechnet dafür $0{,}34\,\%$ (gemessen: $0{,}36\,\%$). Die Analyse dauert für diese Funktion 1,5 ms.

Zeile	Zweig	geschätzt $\hat{p}_b$	gemessen p_b
5	`b != -8` wahr / falsch	0,9950 / 0,0050	0,9950 / 0,0050
7	`a < -39 or t1 != -13` wahr / falsch	0,9916 / 0,0034	0,9914 / 0,0036
12	`while` betreten / verlassen	0,0032 / 0,0034	0,0033 / 0,0036
18	`b > a or a < -31` wahr / falsch	0,5559 / 0,4406	0,5554 / 0,4410
erwartete Coverage $N = 10$ / 100 / 1000		0,518 / 0,656 / 0,986	0,518 / 0,666 / 1,000

Tabelle 5.1: Geschätzte und gemessene Zweigwahrscheinlichkeiten für Listing 5.1. Die gemessene Coverage ist der Mittelwert über 20 Wiederholungen. Die Summe der Ausgänge in Zeile 7 ist kleiner als eins, weil der Zweig nur erreicht wird, wenn `b != -8` gilt.

5.5 Versuchsinfrastruktur

Jeder Versuch ist ein eigenständiges Skript im Verzeichnis `experiments/`, das seine Ergebnisse als CSV- oder JSON-Datei in `data/` ablegt; `plots.py` erzeugt daraus alle Diagramme dieser Arbeit. Die Erzeugung des Hauptdatensatzes ist mit `multiprocessing` über zehn Prozesse parallelisiert. Alle Zufallsentscheidungen werden aus festen Startwerten abgeleitet, sodass jeder Versuch bitgenau wiederholbar ist. Die Befehle zur Reproduktion sind in Anhang A zusammengestellt.

6 Evaluation

Dieses Kapitel beantwortet die Forschungsfragen aus Abschnitt 1.2 experimentell. Nach dem Versuchsaufbau (Abschnitt 6.1) folgen die Ergebnisse zu FF1 bis FF4 und eine Diskussion der Gültigkeit.

6.1 Versuchsaufbau

Datensatz Für den Hauptversuch wurden 5000 Spezifikationen zufällig gezogen: V^* gleichverteilt in $\{2, \dots, 25\}$, Tiefe in $\{1, \dots, \min(6, V^* - 1)\}$, Schleifen in $\{0, \dots, \min(3, V^* - 1)\}$, ein bis vier Parameter, Eingabebereich $R \in \{10, 100, 1000\}$ sowie die weichen Parameter gleichverteilt in plausiblen Intervallen (`p_eq` $\in [0; 0{,}5]$, `p_mod`, `p_rel2` $\in [0; 0{,}3]$, `p_compound`, `p_elif` $\in [0; 0{,}4]$, `p_else` $\in [0{,}2; 0{,}8]$, `p_return` $\in [0; 0{,}3]$). Die erzeugten Funktionen haben 6 bis 145 Codezeilen (Mittel 44) und 2 bis 48 Zweige (Mittel 21,7); insgesamt sind es 108 576 Zweige.

Messung Für jedes Programm wurden 20 Folgen zu je 1000 Zufallstests und 10 Folgen mit Konstanten-Seeding ausgeführt. Die Coverage wurde für die Budgets $N \in \{1, 2, 5, 10, 20, 50, 100, 200, 500, 1000\}$ nach Abschnitt 4.3 bestimmt. Um erreichbare von unerreichten Zweigen zu unterscheiden, wurden zusätzlich je 30 000 Zufallstests mit und ohne Seeding ausgeführt; ein Zweig gilt als *erreicht*, wenn ihn einer der insgesamt 90 000 Tests durchläuft. Der gesamte Versuch dauerte auf einem Apple M1 Max mit zehn Prozessen 96 Sekunden.

Modelle Die lernbasierten Modelle wurden mit fünffacher, nach Programmen gruppierter Kreuzvalidierung bewertet (40 000 Trainings- und 10 000 Testzeilen je Fold). Verwendet wurden lineare Regression mit standardisierten Merkmalen, ein Random Forest mit 300 Bäumen und Gradient Boosting (`HistGradientBoostingRegressor`, 600 Iterationen, Lernrate 0,05). Die Mittelwert-Baseline sagt für jedes Budget die mittlere Coverage der Trainingsdaten voraus.

6.2 FF1: Generierung mit definierten Eigenschaften

Spezifikationstreue Alle 5000 Programme erfüllen alle vier harten Eigenschaften – Komplexität, Tiefe, Schleifen- und Parameterzahl – exakt, geprüft durch die unabhängige Metrikanalyse. Die weichen Eigenschaften folgen ihren Parametern monoton (Abbildung 6.1 b; Pearson-Korrelation zwischen 0,39 und 0,51). Die gemessenen Anteile liegen unter den Parameterwerten, weil Schleifenbedingungen als einfache Vergleiche mitzählen und weil Gleichheitsbedingungen bei den Erfüllbarkeitsprüfungen häufiger verworfen werden: Ihr Wahr-Ausgang legt eine Variable auf einen Wert fest. Die Programmgröße ergibt sich emergent und wächst nahezu linear mit V^* (Abbildung 6.1 a; $r = 0{,}88$).

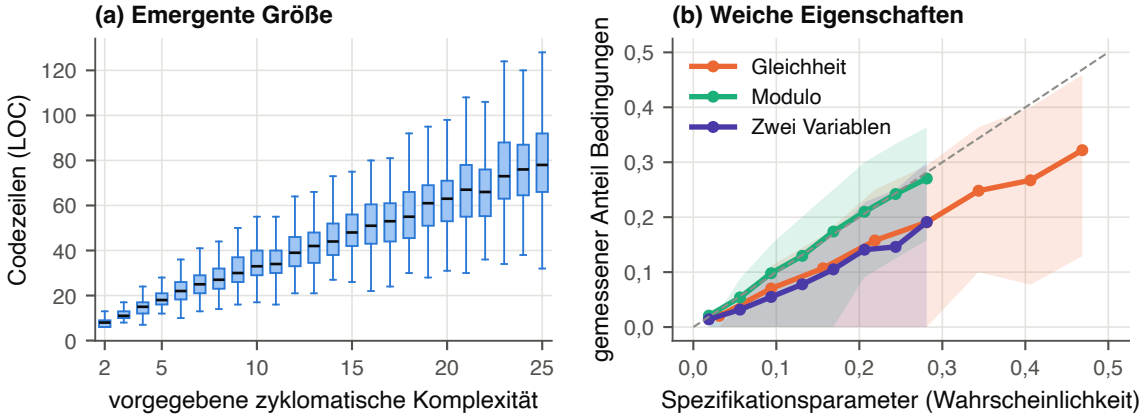


Abbildung 6.1: (a) Codezeilen in Abhängigkeit der vorgegebenen Komplexität; (b) gemessener Anteil der Bedingungsarten über dem jeweiligen Spezifikationsparameter (Mittelwert und Interquartilsbereich; gestrichelt: Identität).

Gültigkeit und Messung In den insgesamt 450 Millionen Testaufrufen trat keine einzige Ausnahme auf, und alle Aufrufe terminierten. Die eigene Zweigmessung stimmt mit coverage.py überein: Für 300 generierte Programme und alle 13 Lehrbuchfunktionen mit je 5 und 50 Zufallstests waren in allen 626 Vergleichen sowohl die Anzahl der Zweige als auch die Anzahl überdeckter Zweige identisch.

Vergleich mit Generate-and-Test Um die Notwendigkeit der konstruktiven Erzeugung zu prüfen, wurden mit einem naiven grammatikbasierten Generator (gleiche Sprachmittel, aber keine Budgetsteuerung) 10^6 Programmstrukturen gezogen. Abbildung 6.2 (a) zeigt, dass die Wahrscheinlichkeitsmasse auf wenige Kombinationen aus Komplexität und Tiefe konzentriert ist. Für 16,6 % der 5000 Spezifikationen trat die geforderte Kombination aus $V(G)$, Tiefe und Schleifenzahl in einer Million Versuchen kein einziges Mal auf, im Median wären rund 2 100 Versuche je Treffer nötig (Abbildung 6.2 b). Für 71 % der Spezifikationen wären mehr als 1000 Versuche erforderlich. Der konstruktive Generator benötigt dagegen genau einen Versuch und im Mittel 2,5 ms (95. Perzentil: 8,9 ms).

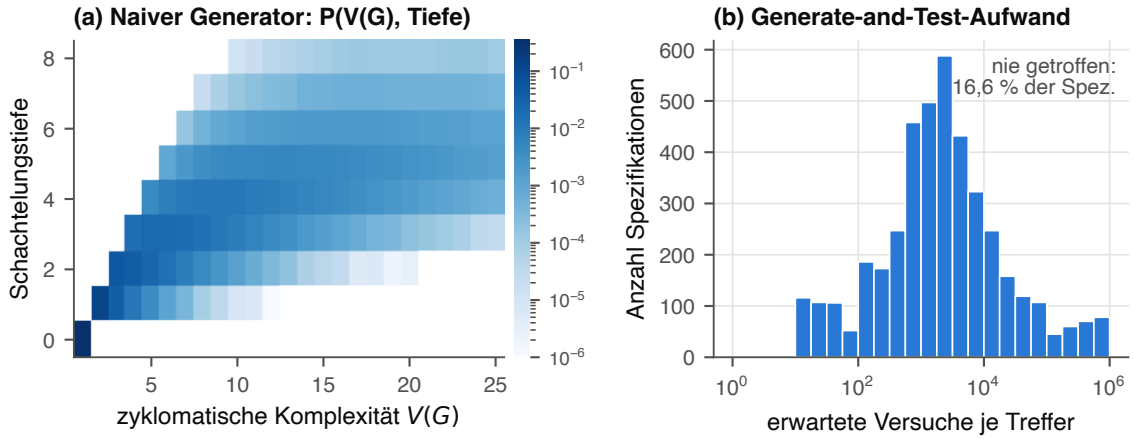


Abbildung 6.2: (a) Verteilung von Komplexität und Tiefe eines naiven grammatikbasierten Generators (10^6 Stichproben, logarithmische Farbskala); (b) erwartete Anzahl Versuche, bis der naive Generator die Strukturvorgaben einer der 5000 Spezifikationen trifft.

Erfüllbarkeit Von allen Zweigen wurden 90,9 % durch mindestens einen der 90 000 Tests erreicht; 59,7 % der Programme sind vollständig überdeckbar. Die übrigen Zweige sind entweder unerreichbar oder so unwahrscheinlich, dass auch Seeding sie nicht trifft. Ursache sind Abhängigkeiten, die die Umgebung des Generators nicht darstellt, etwa zwischen zwei Variablen nach `b < a` oder zwischen einer Variablen und ihrem Modulo-Rest. Eine Vorgängerversion des Generators ohne Ausschlusswerte und Restklassen erreichte nur 86,7 % der Zweige; die Erweiterung hat den Anteil unerreichter Zweige also um rund ein Drittel gesenkt.

Vielfalt Für drei feste Spezifikationen (klein: $V^* = 4$, Tiefe 2; mittel: $V^* = 10$, Tiefe 3; groß: $V^* = 20$, Tiefe 5) wurden je 1000 Programme mit verschiedenen Startwerten erzeugt. Alle 3000 Programme sind verschieden, auch nachdem Konstanten und Bezeichner abstrahiert wurden. Betrachtet man nur die Anordnung der Kontrollstrukturen, ergeben sich bei der kleinen Spezifikation 26 verschiedene Strukturen – mehr lassen die Vorgaben kaum zu –, bei der mittleren und großen jeweils 1000.

6.3 FF2: Einflussfaktoren der Coverage

Im Mittel erreichen Zufallstests nach 1, 10, 100 und 1000 Tests eine Zweigüberdeckung von 30,2 %, 59,6 %, 76,8 % und 85,3 %. Abbildung 6.3 (a) zeigt den starken Einfluss der Komplexität: Einfache Programme ($V(G) \leq 8$) sind im Median nach 100 Tests zu 99,8 % überdeckt, komplexe ($V(G) \geq 17$) erreichen im Median erst nach 1000 Tests rund 86 %. Der Eingabebereich wirkt vor allem über Gleichheitsbedingungen (Abbildung 6.3 b): Bei $N = 100$ sinkt die mittlere Coverage von 81,8 % ($R = 10$) auf 73,4 % ($R = 1000$). Konstanten-

Seeding gleicht diesen Effekt weitgehend aus und hebt die Coverage für $R = 1000$ auf 79,7 %.

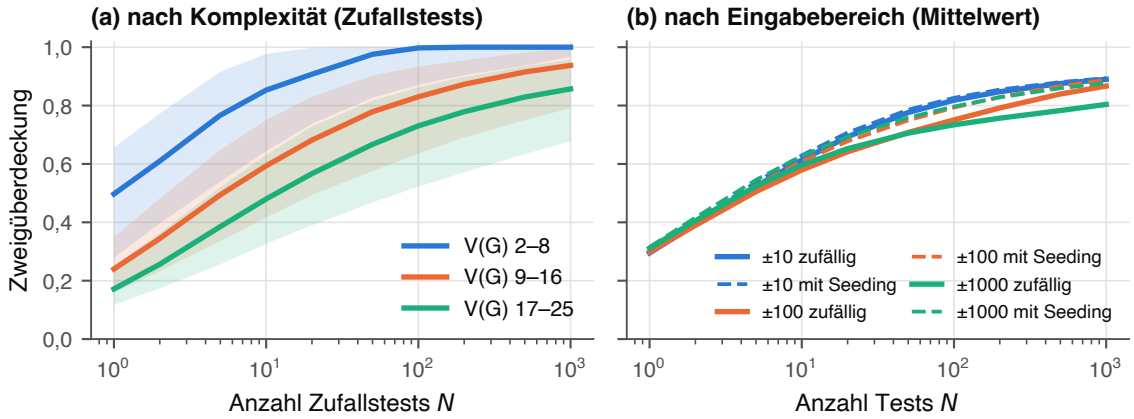


Abbildung 6.3: Gemessene Zweigüberdeckung über dem Testbudget: (a) Median und Interquartilsbereich nach Komplexitätsklassen; (b) Mittelwert nach Eingabebereich für Zufallstests (durchgezogen) und Zufallstests mit Konstanten-Seeding (gestrichelt).

Abbildung 6.4 zeigt die Rangkorrelation einzelner Merkmale mit der Coverage bei $N = 100$. Am stärksten negativ wirken die Anzahl der Gleichheitsbedingungen ($\rho = -0{,}56$), die Gesamtzahl der Zweige, die Komplexität und die Codezeilen (jeweils etwa $-0{,}50$) sowie die Tiefe, in der Entscheidungen im Mittel liegen ($-0{,}49$). Je tiefer ein Zweig geschachtelt ist, desto mehr Bedingungen müssen gleichzeitig gelten und desto kleiner wird p_b. Schleifen haben dagegen kaum Einfluss ($+0{,}06$), weil ihr Rumpf meist mehrfach mit unterschiedlichen Werten der Schleifenvariable durchlaufen wird. Keines der Merkmale erklärt allein mehr als ein Drittel der Rangvarianz ($\rho^2 < 0{,}32$).

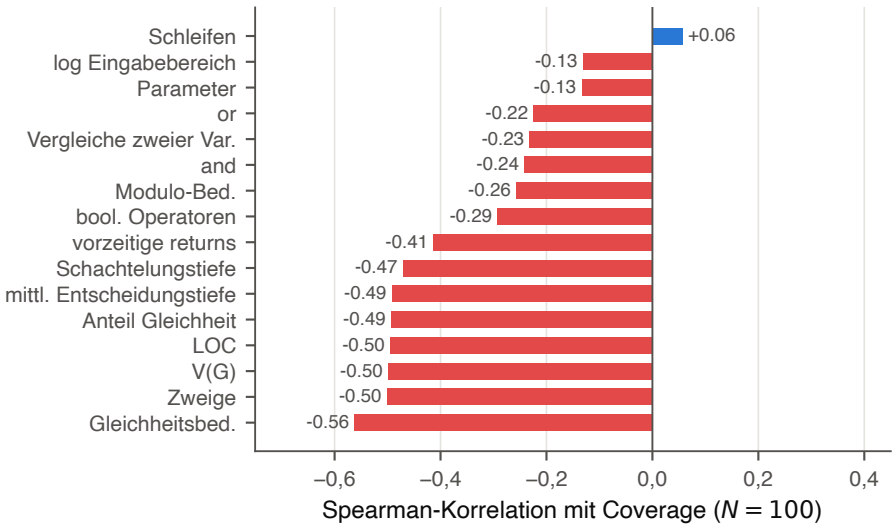


Abbildung 6.4: Spearman-Rangkorrelation statischer Merkmale mit der gemessenen Coverage nach 100 Zufallstests (5000 Programme).

Bemerkenswert ist schließlich die Streuung zwischen einzelnen Testläufen: Die Standardabweichung der Coverage zwischen den 20 Wiederholungen beträgt bei $N = 100$ im Mittel

4,5 Prozentpunkte. Selbst eine perfekte Vorhersage der *erwarteten* Coverage kann das Ergebnis eines einzelnen Testlaufs daher nur auf einige Prozentpunkte genau treffen.

6.4 FF3: Genauigkeit der Vorhersage

Gesamtergebnis Tabelle 6.1 fasst die Kreuzvalidierung über alle Budgets zusammen. Der analytische Schätzer sagt die Coverage mit einem mittleren absoluten Fehler von 0,74 Prozentpunkten voraus ($R^2 = 0{,}990$); in 97 % der Fälle liegt er weniger als fünf Prozentpunkte daneben. Die rein metrikbasierten Modelle erreichen bestenfalls 11,4 Prozentpunkte (PP) (Gradient Boosting, $R^2 = 0{,}711$) und verbessern die Mittelwert-Baseline damit nur um etwa ein Drittel. Abbildung 6.5 zeigt die Unterschiede für $N = 100$.

Modell	MAE [PP]	RMSE [PP]	R^2	$\lvert e\rvert \leq 5$ PP	Spearman
Baseline (Mittelwert je N)	18,54	22,91	0,381	13,9 %	0,595
Lineare Regression	13,17	17,12	0,655	26,3 %	0,828
Random Forest	11,48	15,85	0,704	34,2 %	0,852
Gradient Boosting	11,39	15,65	0,711	33,8 %	0,857
Analytischer Schätzer	**0,74**	2,84	0,990	**97,0 %**	**0,994**
Hybrid (GB + analytisch)	1,02	**2,81**	**0,991**	96,4 %	**0,994**

Tabelle 6.1: Vorhersagegüte bei fünffacher, nach Programmen gruppierter Kreuzvalidierung (5000 Programme, 10 Budgets, 50 000 Zeilen). PP: Prozentpunkte.

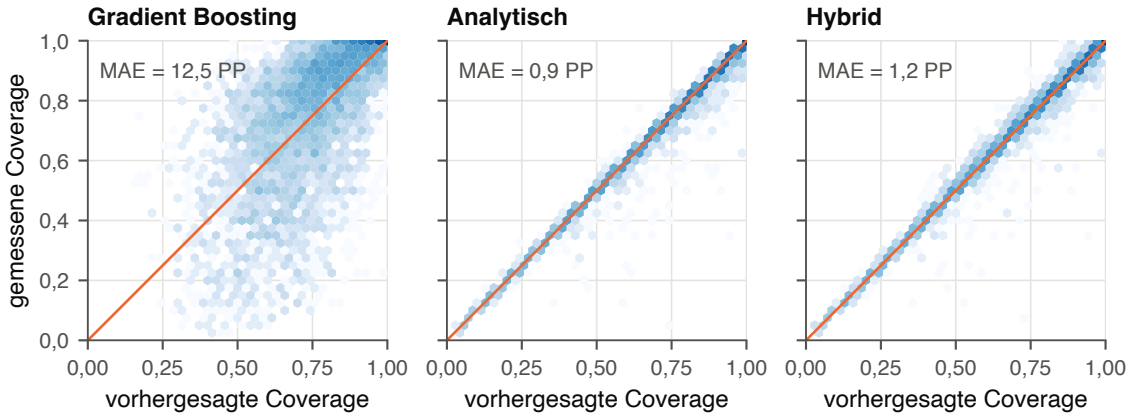


Abbildung 6.5: Vorhergesagte gegenüber gemessener Coverage für $N = 100$ (Kreuzvalidierung; Farbintensität: Anzahl Programme, logarithmisch).

Abhängigkeit vom Budget Der Fehler des Schätzers wächst mit dem Budget von 0,15 Prozentpunkten bei $N = 1$ auf 1,56 bei $N = 1000$ (Abbildung 6.6). Bei kleinen Budgets bestimmen die häufigen Zweige die Coverage, und deren p_b werden genau geschätzt. Bei großen Budgets entscheiden die seltenen Zweige, für die schon kleine absolute Fehler in p_b den Term $1 - (1 - p_b)^N$ stark verändern. Die lernbasierten Modelle haben ihren größten Fehler bei mittleren Budgets, wo die Coverage zwischen Programmen am stärksten streut.

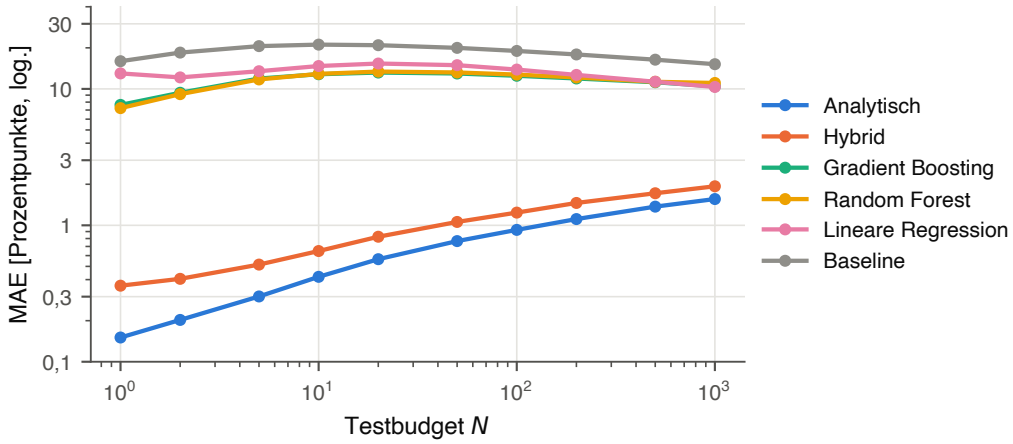


Abbildung 6.6: Mittlerer absoluter Fehler der Modelle je Testbudget (logarithmische Achsen).

Zweigebene Da der Schätzer Wahrscheinlichkeiten je Zweig liefert, lässt er sich auch dort prüfen (Abbildung 6.7). Der mittlere absolute Fehler von $\hat{p}_b$ gegenüber der relativen Trefferhäufigkeit in 20 000 Tests beträgt 0,003; 96,4 % aller Zweige liegen innerhalb von $\pm 0{,}01$, die Rangkorrelation beträgt 0,993. Auch sehr kleine Wahrscheinlichkeiten werden über mehrere Größenordnungen getroffen; das horizontale Band bei $\log_{10} p_b \approx -4{,}3$ entspricht einem einzigen Treffer in 20 000 Tests und ist ein Artefakt der endlichen Stichprobe. Von den nie erreichten Zweigen schätzt der Schätzer 69,5 % mit $\hat{p}_b < 10^{-6}$ als praktisch unerreichbar ein, nur 5,4 % erhalten mehr als 1 %. Umgekehrt werden nur 0,5 % der erreichten Zweige fälschlich als unerreichbar eingestuft.

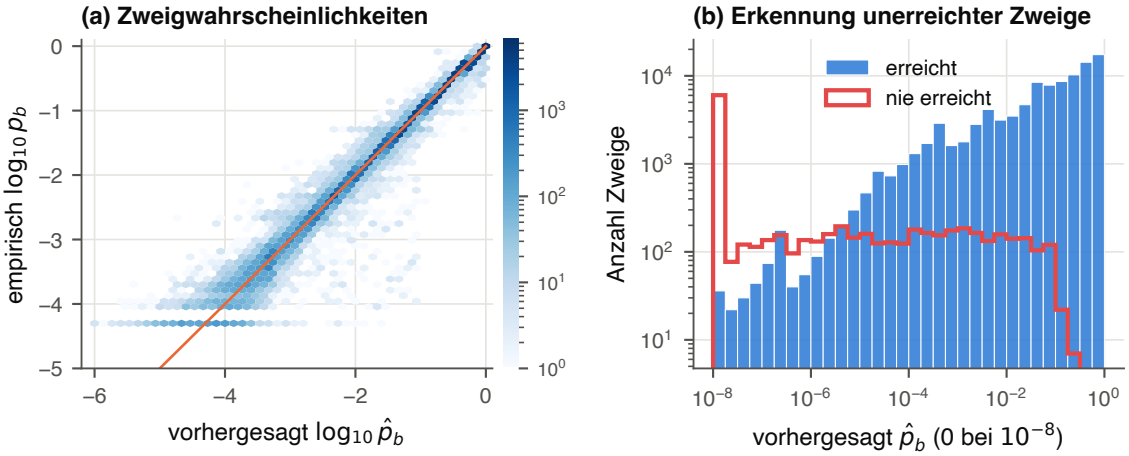


Abbildung 6.7: (a) Geschätzte gegenüber gemessener Trefferwahrscheinlichkeit erreichter Zweige; (b) Verteilung der Schätzwerte für erreichte und nie erreichte Zweige.

Fehlerursachen Die Fehler des Schätzers sind stark konzentriert: Bei $N = 100$ weichen nur 4,0 % der Programme um mehr als fünf und 1,5 % um mehr als zehn Prozentpunkte ab. In den 9,0 % der Programme, die einen nie erreichten, aber mit $\hat{p}_b > 10^{-3}$ geschätzten Zweig enthalten, beträgt der Fehler im Mittel 6,1 Prozentpunkte, in allen übrigen nur 0,41. Hauptursache ist damit die Mean-Field-Näherung: Nach Bedingungen über mehrere

Variablen gehen Korrelationen verloren, sodass unmögliche Kombinationen eine kleine positive Wahrscheinlichkeit erhalten. Größe und Eingabebereich der Programme spielen dagegen kaum eine Rolle (Fehler zwischen 0,6 und 1,1 Prozentpunkten in allen Klassen).

Lernbasierte Modelle Für das beste lernbasierte Modell zeigt die Permutationswichtigkeit (Abbildung 6.8), dass das Budget mit Abstand das wichtigste Merkmal ist, gefolgt von der mittleren Entscheidungstiefe, der Anzahl der `if`-Anweisungen, vorzeitigen Rücksprüngen und Gleichheitsbedingungen. Die Lernkurve (Abbildung 6.9) flacht ab etwa 1000 Trainingsprogrammen ab; mit 4000 Programmen sinkt der Fehler des Gradient Boosting nur noch auf 11,2 Prozentpunkte. Mehr Daten helfen also kaum: Statischen Metriken fehlt die entscheidende Information, nämlich wie Konstanten, Wertebereiche und Abhängigkeiten zwischen Variablen zueinander liegen. Zwei Programme mit identischen Metriken können sich in ihrer Coverage um Dutzende Prozentpunkte unterscheiden, je nachdem ob eine Bedingung `x > 0` oder `x == 4711` lautet.

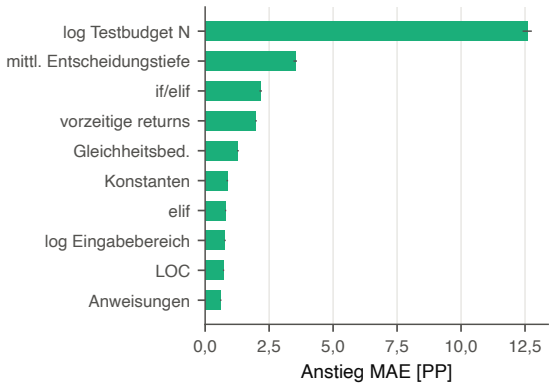


Abbildung 6.8: Permutationswichtigkeit der zehn wichtigsten Merkmale (Gradient Boosting, 10 Wiederholungen).

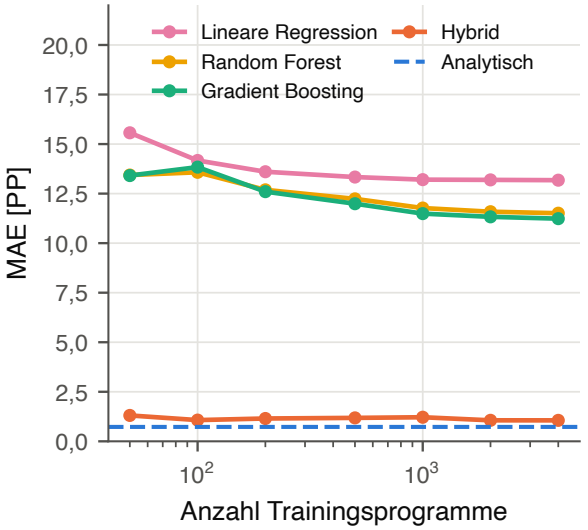


Abbildung 6.9: Fehler über der Anzahl der Trainingsprogramme (feste Testmenge aus 1000 Programmen).

Hybridmodell Das Hybridmodell erreicht das höchste Bestimmtheitsmaß und den kleinsten quadratischen Fehler, sein mittlerer absoluter Fehler liegt mit 1,02 Prozentpunkten aber über dem des Schätzers allein. Es korrigiert die leichte systematische Überschätzung des Schätzers (mittlere Abweichung $-0{,}04$ statt $+0{,}42$ Prozentpunkte), fügt bei den vielen bereits sehr genau geschätzten Programmen aber Rauschen hinzu. Die statischen Merkmale enthalten offenbar zu wenig Information, um die wenigen großen Fehler des Schätzers vorherzusehen.

Laufzeit Der Schätzer benötigt im Median 4,3 ms je Programm (95. Perzentil: 38 ms). Einzelne Programme mit mehreren geschachtelten Schleifen und breiten Verteilungen dauern

bis zu 7 s, weil das Abrollen viele Faltungen erfordert. Die Messung von 1000 Tests dauert für typische Programme wenige Millisekunden; der Vorteil der Vorhersage liegt daher weniger in der Rechenzeit als darin, dass sie ohne Ausführung, für jedes Budget und für jeden einzelnen Zweig verfügbar ist.

6.5 FF3: Übertragbarkeit

Drei Szenarien prüfen, wie gut die Verfahren außerhalb ihrer Trainingsverteilung funktionieren (Abbildung 6.10): Training auf Programmen mit $V(G) \leq 12$ und Test auf $V(G) \geq 13$, Training auf $R \leq 100$ und Test auf $R = 1000$ sowie Training auf allen 5000 generierten Programmen und Test auf den 13 handgeschriebenen Lehrbuchfunktionen.

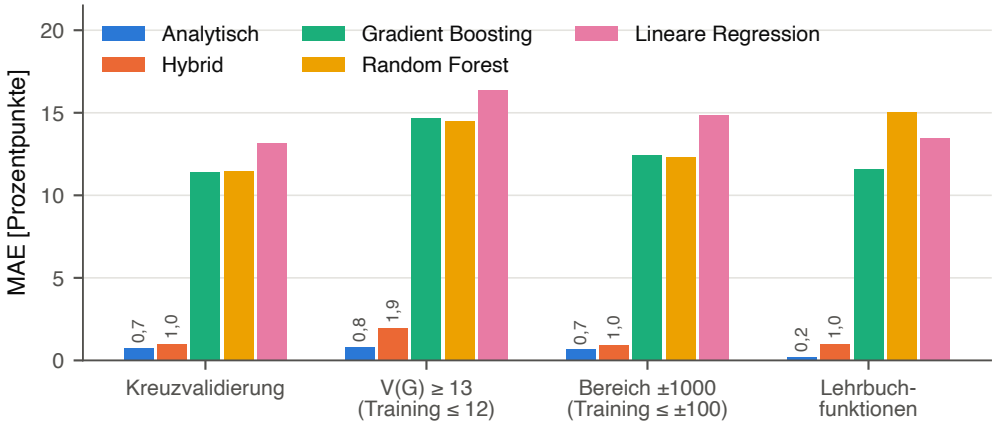


Abbildung 6.10: Mittlerer absoluter Fehler in der Kreuzvalidierung und in drei Übertragungsszenarien. Die Werte des analytischen Schätzers und des Hybridmodells sind beschriftet.

Die lernbasierten Modelle verschlechtern sich bei der Extrapolation in der Komplexität auf 14,5 bis 16,4 Prozentpunkte und werden systematisch verzerrt; die lineare Regression unterschätzt die Coverage komplexer Programme im Mittel um 7,7 Prozentpunkte. Der analytische Schätzer benötigt kein Training und bleibt mit 0,84 bzw. 0,66 Prozentpunkten so genau wie in der Kreuzvalidierung. Das Hybridmodell erbt einen Teil der Verzerrung (1,94 Prozentpunkte bei der Komplexitätsextrapolation).

Tabelle 6.2 zeigt die Ergebnisse für die Lehrbuchfunktionen. Sie enthalten Konstrukte, die der Generator nicht erzeugt, etwa Tupelzuweisungen (`a, b = b, a % b`), ganzzahlige Division, verschachtelte boolesche Ausdrücke mit Klammern, Schleifen mit Rücksprung und `while`-Schleifen mit datenabhängiger Abbruchbedingung. Der analytische Schätzer sagt die Coverage über alle Budgets mit einem mittleren Fehler von 0,21 Prozentpunkten voraus; die größte Abweichung tritt bei `triangle` auf, wo drei Variablen über Summen verknüpft sind. Gradient Boosting unterschätzt die Coverage dieser im Vergleich zum Trainingsdatensatz einfach zu testenden Funktionen dagegen im Mittel um 10,3 Prozentpunkte (MAE 11,6).

			$N = 10$			$N = 100$		
Funktion	$V(G)$	**Zweige**	**gemessen**	**analyt.**	**GB**	**gemessen**	**analyt.**	**GB**
triangle	12	8	0,818	0,807	0,562	0,916	0,901	0,797
leap_year	4	6	0,674	0,672	0,760	0,802	0,791	0,949
days_in_month	11	8	0,734	0,733	0,492	0,982	0,978	0,608
grade	7	10	0,850	0,849	0,680	1,000	1,000	0,827
fizzbuzz	4	6	0,866	0,866	0,658	1,000	1,000	0,866
gcd	4	6	1,000	0,999	0,822	1,000	1,000	0,902
is_prime	5	8	0,762	0,763	0,758	0,903	0,908	0,930
quadrant	6	8	0,914	0,914	0,682	1,000	1,000	0,879
ticket_price	8	10	0,891	0,894	0,737	1,000	1,000	0,902
tax_class	5	8	0,786	0,785	0,722	0,994	0,997	0,862
collatz_steps	4	4	1,000	1,000	0,784	1,000	1,000	0,938
max3	3	4	0,995	0,995	0,930	1,000	1,000	0,984
digit_sum	3	4	1,000	1,000	0,953	1,000	1,000	1,000

Tabelle 6.2: Gemessene (Mittel aus 20 Wiederholungen) und vorhergesagte Zweigüberdeckung für 13 handgeschriebene Lehrbuchfunktionen. GB: Gradient Boosting, trainiert auf den 5000 generierten Programmen.

6.6 FF4: Coverage als Zieleigenschaft

Für sechs Zielwerte $C^* \in \{0{,}5; 0{,}6; \ldots; 1{,}0\}$ wurden mit dem Verfahren aus Abschnitt 4.6 je 60 Programme für $N = 100$ und $\varepsilon = 0{,}025$ erzeugt und anschließend mit 20 Wiederholungen gemessen. Ohne Filter streut die Coverage der Programme zwischen 3 % und 100 % (Abbildung 6.11). Mit Filter liegt die mittlere gemessene Coverage in allen Gruppen höchstens einen Prozentpunkt vom Ziel entfernt (Tabelle 6.3), die mittlere absolute Abweichung einzelner Programme beträgt 0,5 bis 2,4 Prozentpunkte. Einzelne Ausreißer nach unten, etwa ein Programm mit 21 % statt 50 %, gehen auf die oben beschriebenen fälschlich als erreichbar geschätzten Zweige zurück. Niedrige Zielwerte sind seltener und erfordern mehr Kandidaten; da ein Kandidat nur Millisekunden kostet, dauerte selbst die Erzeugung von 60 Programmen mit $C^* = 0{,}5$ einschließlich Messung nur 26 Sekunden.

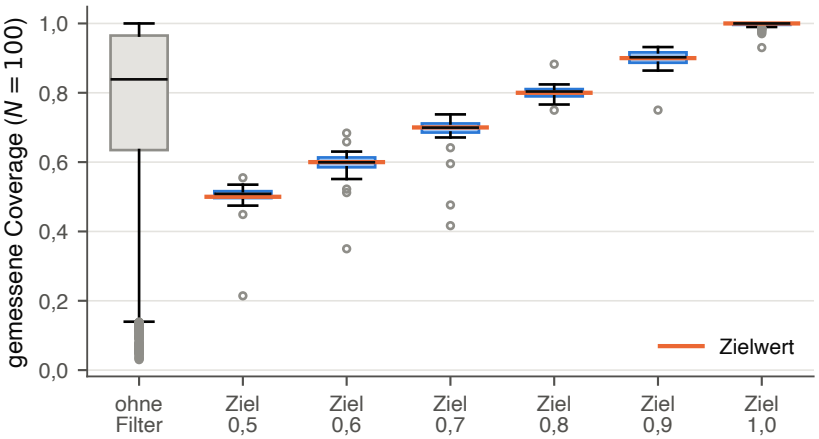


Abbildung 6.11: Gemessene Coverage ($N = 100$) von 5000 ungefilterten Programmen und von je 60 Programmen, die mit der analytischen Vorhersage auf einen Zielwert gefiltert wurden.

Ziel C^*	Kandidaten	Akzeptanz	gemessen (Mittel)	MAE [PP]	Zeit [s]
0,5	2252	2,7 %	0,501	1,99	25,8
0,6	1458	4,1 %	0,596	2,15	16,0
0,7	1284	4,7 %	0,690	2,35	14,1
0,8	799	7,5 %	0,802	1,36	12,6
0,9	633	9,5 %	0,899	1,74	7,3
1,0	227	26,4 %	0,995	0,47	2,7

Tabelle 6.3: Coverage-gesteuerte Generierung von je 60 Programmen ($N = 100$, Toleranz $\pm 0{,}025$). Die Zeit umfasst Generierung, Vorhersage und Messung.

6.7 Diskussion und Gültigkeit

Zusammenfassung der Antworten *FF1:* Die konstruktive Erzeugung erfüllt harte Strukturvorgaben ausnahmslos, in Millisekunden und mit hoher Vielfalt, während Generate-and-Test für einen großen Teil der Vorgaben praktisch versagt. Rund 9 % der Zweige bleiben unerreicht, weil nicht alle Abhängigkeiten zwischen Variablen verfolgt werden. *FF2:* Die erreichbare Coverage hängt vor allem von Gleichheitsbedingungen, der Komplexität und der Tiefe der Entscheidungen ab; diese Merkmale erklären sie aber nur teilweise. *FF3:* Eine Vorhersage aus Werteverteilungen ist um mehr als eine Größenordnung genauer als eine Vorhersage aus statischen Metriken und überträgt sich auf ungesehene Programme. *FF4:* Die Vorhersage macht die erwartete Coverage zu einer steuerbaren Eigenschaft generierten Codes.

Interne Validität Generator und Schätzer stammen vom selben Autor, sodass der Schätzer unbewusst auf die Konstrukte des Generators zugeschnitten sein könnte. Dem wirken die vom Generator unabhängige Analyse, die Validierung gegen coverage.py und die Fallstudie mit handgeschriebenen Funktionen entgegen. Die gemessene Coverage ist selbst ein Schätzwert; durch 20 Wiederholungen und die Mittelung über Teilfolgen ist ihr Standardfehler aber deutlich kleiner als die berichteten Unterschiede zwischen den Verfahren.

Externe Validität Die Ergebnisse gelten für Funktionen mit ganzzahligen Parametern und gleichverteilten Zufallstests. Realer Code enthält Datenstrukturen, Zeichenketten, Objekte, Aufrufe anderer Funktionen und Ausnahmen, für die der Schätzer erweitert werden müsste; die Mean-Field-Näherung dürfte dort häufiger an ihre Grenzen stoßen. Die lernbasierten Modelle sind auf beliebigen Code anwendbar, profitieren aber nur, wenn ihre Merkmale die wertebezogenen Ursachen schwer erreichbarer Zweige abbilden. Für suchbasierte oder symbolische Testverfahren gilt Gleichung (2.4) nicht unmittelbar, da deren Tests nicht unabhängig sind.

Konstruktvalidität Zweigüberdeckung ist nur ein Aspekt der Testgüte (vgl. Abschnitt 2.1). Vorhergesagt wird zudem die *erwartete* Coverage; für einen einzelnen Testlauf kommt die Streuung von im Mittel 4,5 Prozentpunkten bei $N = 100$ hinzu.

7 Zusammenfassung und Ausblick

7.1 Zusammenfassung

Diese Arbeit hat untersucht, wie sich Beispielcode mit definierten Eigenschaften erzeugen und die Testüberdeckung solchen Codes vorhersagen lässt. Dazu wurde ein Eigenschaftsmodell entwickelt, das harte Strukturvorgaben – zyklomatische Komplexität, Schachtelungstiefe, Schleifen- und Parameterzahl, Eingabebereich – von weichen, verteilungsbasierten Vorgaben zur Art der Bedingungen trennt.

Der konstruktive Generator erfüllt die harten Vorgaben per Konstruktion: Das Entscheidungsbudget wird so verteilt, dass jede Einheit genau einen Beitrag zur Komplexität liefert, und ein Rückgrat geschachtelter Knoten garantiert die geforderte Tiefe. Während des Renderings führt der Generator Intervalle, Ausschlusswerte und Restklassen mit, damit Bedingungen erfüllbar bleiben. In der Evaluation erfüllten alle 5000 Programme ihre harten Vorgaben, kein einziger der 450 Millionen Testaufrufe führte zu einem Fehler, und 90,9 % aller Zweige waren erreichbar. Ein naiver Generate-and-Test-Ansatz hätte dagegen für 16,6 % der Vorgaben in einer Million Versuchen keinen Treffer erzielt.

Zur Vorhersage der Coverage zufälliger Tests wurde ein analytischer Schätzer entwickelt, der diskrete Werteverteilungen über lineare Formen propagiert, an Verzweigungen exakt bedingt und Schleifen symbolisch abrollt. Er sagt die Zweigüberdeckung im Mittel auf 0,74 Prozentpunkte genau voraus, liefert kalibrierte Wahrscheinlichkeiten für einzelne Zweige und überträgt sich ohne Anpassung auf handgeschriebene Lehrbuchfunktionen (0,21 Prozentpunkte). Lernbasierte Modelle auf statischen Code-Metriken kamen dagegen nicht unter 11 Prozentpunkte und verschlechterten sich außerhalb ihrer Trainingsverteilung deutlich. Die zentrale Erkenntnis lautet: Wie gut Zufallstests einen Code überdecken, hängt weniger von seiner Größe oder Komplexität ab als von der Lage seiner Konstanten relativ zu den Wertebereichen der Variablen – einer Information, die klassische Metriken nicht enthalten.

Schließlich wurde gezeigt, dass die Vorhersage die erwartete Coverage selbst zu einer steuerbaren Eigenschaft macht. Programme, die auf eine Ziel-Coverage gefiltert wurden, trafen diese im Mittel auf einen Prozentpunkt genau.

7.2 Ausblick

Aus den Ergebnissen ergeben sich mehrere Ansatzpunkte für weiterführende Arbeiten:

- **Relationale Abstraktionen.** Sowohl die verbleibenden unerreichbaren Zweige des Generators als auch die größten Fehler des Schätzers gehen auf nicht verfolgte Abhängigkeiten zwischen Variablen zurück. Relationale Bereiche wie Oktagone oder Polyeder könnten beide Schwächen gleichzeitig verringern.
- **Weitere Teststrategien.** Da der Schätzer Eingaben als unabhängige Verteilungen behandelt, lässt sich Konstanten-Seeding direkt als Mischverteilung modellieren. Für suchbasierte Verfahren wie Pynguin wäre ein Modell der Suchdynamik nötig, etwa über eine Kombination aus Zweigwahrscheinlichkeiten und Fitnesslandschaft.
- **Weitere Überdeckungskriterien.** Die Wahrscheinlichkeiten atomarer Bedingungen liegen bereits vor; daraus ließen sich Bedingungs- und MC/DC-Überdeckung vorhersagen.
- **Mächtigere Sprachmittel.** Listen, Zeichenketten, Gleitkommazahlen und Funktionsaufrufe würden den Generator für realistischere Übungsaufgaben öffnen, erfordern aber passende abstrakte Bereiche.
- **Kombination mit Sprachmodellen.** Ein großes Sprachmodell könnte die von ppgen erzeugten Strukturen mit sprechenden Namen und einer fachlichen Bedeutung versehen; die unabhängige Metrikanalyse und der Schätzer prüfen anschließend, ob die Eigenschaften erhalten geblieben sind.
- **Einsatz in der Lehre.** Mit vorhersagbarer Coverage lassen sich Übungsaufgaben mit kalibriertem Schwierigkeitsgrad erzeugen, etwa Funktionen, bei denen Zufallstests erkennbar an Gleichheitsbedingungen scheitern und Studierende gezielt Grenzwerte ergänzen müssen.

Abkürzungsverzeichnis

FF	Forschungsfrage	2
KFG	Kontrollflussgraph	4
LOC	Lines of Code (Codezeilen)	6
MAE	Mean Absolute Error (mittlerer absoluter Fehler)	9
MC/DC	Modified Condition/Decision Coverage	5
PMF	Probability Mass Function (Wahrscheinlichkeitsfunktion)	18
PP	Prozentpunkte	29
RMSE	Root Mean Squared Error (Wurzel des mittleren quadratischen Fehlers)	9

Tabellenverzeichnis

3.1 Vergleich verwandter Ansätze 11

4.1 Parameter des Eigenschaftsmodells 14
4.2 Kombination von Trefferwahrscheinlichkeiten über Iterationen 19

5.1 Geschätzte und gemessene Zweigwahrscheinlichkeiten 24

6.1 Vorhersagegüte der Modelle 29
6.2 Coverage-Vorhersage für Lehrbuchfunktionen 33
6.3 Ergebnisse der coverage-gesteuerten Generierung 34

Abbildungsverzeichnis

1.1 Überblick über das Vorgehen 2

2.1 Kontrollflussgraph und Überdeckungskriterien 5
2.2 Überdeckungswahrscheinlichkeit eines Zweigs 7

4.1 Budgetverteilung und Kontrollbaum 15
4.2 Verfeinerung der Wertemenge entlang eines Pfades 16
4.3 Propagation diskreter Verteilungen 19

5.1 Architektur des Prototyps 21

6.1 Emergente und weiche Eigenschaften 26
6.2 Generate-and-Test im Vergleich 27
6.3 Coverage-Kurven nach Komplexität und Eingabebereich 28
6.4 Korrelation von Code-Merkmalen mit der Coverage 28
6.5 Vorhergesagte und gemessene Coverage 29
6.6 Vorhersagefehler über dem Testbudget 30
6.7 Kalibrierung der Zweigwahrscheinlichkeiten 30
6.8 Permutationswichtigkeit der Merkmale 31
6.9 Lernkurve 31
6.10 Übertragbarkeit der Vorhersagemodelle 32
6.11 Coverage-gesteuerte Generierung 33

Quellcodeverzeichnis

2.1 Beispielfunktion mit vier Entscheidungen 4

5.1 Vom Generator erzeugte Funktion (Startwert 10) 22
5.2 Kern der Zweig-Instrumentierung (gekürzt) 23

A.1 Befehle zur Reproduktion ix

B.1 Instrumentierte Fassung von Listing 5.1 x

Literatur

[AIB12] Arcuri, A.; Iqbal, M. Z.; Briand, L.: Random Testing: Theoretical Results and Practical Implications. IEEE Transactions on Software Engineering 38 (2), S. 258–277, 2012.

[AO16] Ammann, P.; Offutt, J.: Introduction to Software Testing. Cambridge University Press, Cambridge, 2016.

[Au21] Austin, J. et al.: Program Synthesis with Large Language Models, 2021, arXiv: 2108.07732.

[Ba26] Batchelder, N. et al.: Coverage.py, Version 7.16.2, 2026, https://coverage.readthedocs.io/, Stand: 16. 10. 2026.

[BL93] Ball, T.; Larus, J. R.: Branch Prediction for Free. In: Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI). ACM, S. 300–313, 1993.

[Bö18] Böhme, M.: STADS: Software Testing as Species Discovery. ACM Transactions on Software Engineering and Methodology 27 (2), Artikel 7, 2018.

[BPR16] Böhme, M.; Pham, V.-T.; Roychoudhury, A.: Coverage-Based Greybox Fuzzing as Markov Chain. In: Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS). ACM, S. 1032–1043, 2016.

[Br01] Breiman, L.: Random Forests. Machine Learning 45 (1), S. 5–32, 2001.

[CC77] Cousot, P.; Cousot, R.: Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints. In: Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL). ACM, S. 238–252, 1977.

[CDE08] Cadar, C.; Dunbar, D.; Engler, D.: KLEE: Unassisted and Automatic Generation of High-Coverage Tests for Complex Systems Programs. In: Proceedings of the 8th USENIX Symposium on Operating Systems Design and Implementation (OSDI). USENIX Association, S. 209–224, 2008.

[CH00] Claessen, K.; Hughes, J.: QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs. In: Proceedings of the 5th ACM SIGPLAN International Conference on Functional Programming (ICFP). ACM, S. 268–279, 2000.

[Ch10] Chen, T. Y. et al.: Adaptive Random Testing: The ART of Test Case Diversity. Journal of Systems and Software 83 (1), S. 60–66, 2010.

[Ch21] Chen, M. et al.: Evaluating Large Language Models Trained on Code, 2021, arXiv: 2107.03374.

[CM94] Chilenski, J. J.; Miller, S. P.: Applicability of Modified Condition/Decision Coverage to Software Testing. Software Engineering Journal 9 (5), S. 193–200, 1994.

[DN84] Duran, J. W.; Ntafos, S. C.: An Evaluation of Random Testing. IEEE Transactions on Software Engineering SE-10 (4), S. 438–444, 1984.

[FA11] Fraser, G.; Arcuri, A.: EvoSuite: Automatic Test Suite Generation for Object-Oriented Software. In: Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (ESEC/FSE). ACM, S. 416–419, 2011.

[FA12] Fraser, G.; Arcuri, A.: The Seed is Strong: Seeding Strategies in Search-Based Software Testing. In: Proceedings of the 5th IEEE International Conference on Software Testing, Verification and Validation (ICST). IEEE, S. 121–130, 2012.

[FCA13] Ferrer, J.; Chicano, F.; Alba, E.: Estimating Software Testing Complexity. Information and Software Technology 55 (12), S. 2125–2139, 2013.

[FPV13] Filieri, A.; Păsăreanu, C. S.; Visser, W.: Reliability Analysis in Symbolic PathFinder. In: Proceedings of the 35th International Conference on Software Engineering (ICSE). IEEE, S. 622–631, 2013.

[Fr01] Friedman, J. H.: Greedy Function Approximation: A Gradient Boosting Machine. The Annals of Statistics 29 (5), S. 1189–1232, 2001.

[GDV12] Geldenhuys, J.; Dwyer, M. B.; Visser, W.: Probabilistic Symbolic Execution. In: Proceedings of the International Symposium on Software Testing and Analysis (ISSTA). ACM, S. 166–176, 2012.

[GJG14] Gopinath, R.; Jensen, C.; Groce, A.: Code Coverage for Suite Evaluation by Developers. In: Proceedings of the 36th International Conference on Software Engineering (ICSE). ACM, S. 72–82, 2014.

[GKS05] Godefroid, P.; Klarlund, N.; Sen, K.: DART: Directed Automated Random Testing. In: Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI). ACM, S. 213–223, 2005.

[GPS17] Gulwani, S.; Polozov, O.; Singh, R.: Program Synthesis. Foundations and Trends in Programming Languages 4 (1–2), S. 1–119, 2017.

[Gr12] Groce, A. et al.: Swarm Testing. In: Proceedings of the International Symposium on Software Testing and Analysis (ISSTA). ACM, S. 78–88, 2012.

[Gr18] Grano, G. et al.: How High Will It Be? Using Machine Learning Models to Predict Branch Coverage in Automated Testing. In: Proceedings of the IEEE Workshop on Machine Learning Techniques for Software Quality Evaluation (MaLTeSQuE). IEEE, 2018.

[Gr19] Grano, G. et al.: Branch Coverage Prediction in Automated Testing. Journal of Software: Evolution and Process 31 (9), Artikel e2158, 2019.

[Ha70] Hanford, K. V.: Automatic Generation of Test Cases. IBM Systems Journal 9 (4), S. 242–257, 1970.

[Ha77] Halstead, M. H.: Elements of Software Science. Elsevier North-Holland, New York, 1977.

[HHZ12] Holler, C.; Herzig, K.; Zeller, A.: Fuzzing with Code Fragments. In: Proceedings of the 21st USENIX Security Symposium. USENIX Association, S. 445–458, 2012.

[HTF09] Hastie, T.; Tibshirani, R.; Friedman, J.: The Elements of Statistical Learning: Data Mining, Inference, and Prediction. Springer, New York, 2009.

[IH14] Inozemtseva, L.; Holmes, R.: Coverage Is Not Strongly Correlated with Test Suite Effectiveness. In: Proceedings of the 36th International Conference on Software Engineering (ICSE). ACM, S. 435–445, 2014.

[JH11] Jia, Y.; Harman, M.: An Analysis and Survey of the Development of Mutation Testing. IEEE Transactions on Software Engineering 37 (5), S. 649–678, 2011.

[Ke17] Ke, G. et al.: LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In: Advances in Neural Information Processing Systems 30 (NIPS). S. 3146–3154, 2017.

[Ki76] King, J. C.: Symbolic Execution and Program Testing. Communications of the ACM 19 (7), S. 385–394, 1976.

[Ko95] Kohavi, R.: A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection. In: Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI). Morgan Kaufmann, S. 1137–1143, 1995.

[LBR20] Livinskii, V.; Babokin, D.; Regehr, J.: Random Testing for C and C++ Compilers with YARPGen. Proceedings of the ACM on Programming Languages 4 (OOPSLA), Artikel 196, 2020.

[Le23] Lemieux, C. et al.: CodaMosa: Escaping Coverage Plateaus in Test Generation with Pre-trained Large Language Models. In: Proceedings of the 45th International Conference on Software Engineering (ICSE). IEEE, S. 919–931, 2023.

[LF22] Lukasczyk, S.; Fraser, G.: Pynguin: Automated Unit Test Generation for Python. In: Proceedings of the 44th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion). ACM, 2022.

[Li09] Liggesmeyer, P.: Software-Qualität: Testen, Analysieren und Verifizieren von Software. Spektrum Akademischer Verlag, Heidelberg, 2009.

[Li23] Liyanage, D. et al.: Reachable Coverage: Estimating Saturation in Fuzzing. In: Proceedings of the 45th International Conference on Software Engineering (ICSE). IEEE, 2023.

[LKF23] Lukasczyk, S.; Kroiß, F.; Fraser, G.: An Empirical Study of Automated Unit Test Generation for Python. Empirical Software Engineering 28 (2), Artikel 36, 2023.

[Mc04] McMinn, P.: Search-Based Software Test Data Generation: A Survey. Software Testing, Verification and Reliability 14 (2), S. 105–156, 2004.

[Mc76] McCabe, T. J.: A Complexity Measure. IEEE Transactions on Software Engineering SE-2 (4), S. 308–320, 1976.

[MH+19] MacIver, D. R.; Hatfield-Dodds, Z. et al.: Hypothesis: A New Approach to Property-Based Testing. Journal of Open Source Software 4 (43), S. 1891, 2019.

[Mo00] Monniaux, D.: Abstract Interpretation of Probabilistic Semantics. In: Static Analysis, 7th International Symposium (SAS). Bd. 1824. Lecture Notes in Computer Science, Springer, S. 322–339, 2000.

[MSB11] Myers, G. J.; Sandler, C.; Badgett, T.: The Art of Software Testing. John Wiley & Sons, Hoboken, NJ, 2011.

[My77] Myers, G. J.: An Extension to the Cyclomatic Measure of Program Complexity. ACM SIGPLAN Notices 12 (10), S. 61–64, 1977.

[Pa07] Pacheco, C. et al.: Feedback-Directed Random Test Generation. In: Proceedings of the 29th International Conference on Software Engineering (ICSE). IEEE, S. 75–84, 2007.

[Pa11] Pałka, M. H. et al.: Testing an Optimising Compiler by Generating Random Lambda Terms. In: Proceedings of the 6th International Workshop on Automation of Software Test (AST). ACM, S. 91–97, 2011.

[Pe11] Pedregosa, F. et al.: Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research 12, S. 2825–2830, 2011.

[Pu72] Purdom, P.: A Sentence Generator for Testing Parsers. BIT Numerical Mathematics 12 (3), S. 366–375, 1972.

[Py26] Python Software Foundation: ast – Abstract Syntax Trees, 2026, https://docs.python.org/3/library/ast.html, Stand: 16. 10. 2026.

[RT11] RTCA: DO-178C: Software Considerations in Airborne Systems and Equipment Certification, RTCA, Inc., Washington, DC, 2011.

[Sa22] Sarsa, S. et al.: Automatic Generation of Programming Exercises and Code Explanations Using Large Language Models. In: Proceedings of the ACM Conference on International Computing Education Research (ICER). ACM, S. 27–43, 2022.

[Sc24] Schäfer, M. et al.: An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation. IEEE Transactions on Software Engineering 50 (1), S. 85–105, 2024.

[Vo92] Voas, J. M.: PIE: A Dynamic Failure-Based Technique. IEEE Transactions on Software Engineering 18 (8), S. 717–727, 1992.

[WM96] Watson, A. H.; McCabe, T. J.: Structured Testing: A Testing Methodology Using the Cyclomatic Complexity Metric, NIST Special Publication 500-235, National Institute of Standards und Technology, 1996.

[Ya11] Yang, X. et al.: Finding and Understanding Bugs in C Compilers. In: Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI). ACM, S. 283–294, 2011.

[Ze24] Zeller, A. et al.: The Fuzzing Book: Tools and Techniques for Generating Software Tests, CISPA Helmholtz Center for Information Security, 2024, https://www.fuzzingbook.org/, Stand: 16. 10. 2026.

[ZHM97] Zhu, H.; Hall, P. A. V.; May, J. H. R.: Software Unit Test Coverage and Adequacy. ACM Computing Surveys 29 (4), S. 366–427, 1997.

A Reproduktion der Versuche

Der Prototyp liegt im Verzeichnis `prototyp/` neben dieser Arbeit. Alle Versuche sind deterministisch; die folgenden Befehle erzeugen sämtliche Daten in `prototyp/data/` und alle Diagramme in `thesis/bilder/plots/`. Die Laufzeitangaben beziehen sich auf einen Apple M1 Max mit zehn Kernen.

```
cd prototyp
python3 -m venv .venv
.venv/bin/pip install numpy scipy pandas scikit-learn matplotlib
    coverage
.venv/bin/python experiments/exp_dataset.py 5000 10 # Versuch 1, ca. 2
    min
.venv/bin/python experiments/exp_naive.py       # Versuch 3
.venv/bin/python experiments/exp_casestudy.py   # Versuch 4
.venv/bin/python experiments/exp_coveragepy.py  # Versuch 2
.venv/bin/python experiments/exp_models.py      # Versuch 5, ca. 10 min
.venv/bin/python experiments/exp_targeted.py    # Versuch 6
.venv/bin/python experiments/exp_diversity.py   # Versuch 7
.venv/bin/python experiments/exp_stats.py       # Kennzahlen
.venv/bin/python experiments/plots.py           # Abbildungen
cd ../thesis && make                            # Arbeit (LuaLaTeX)
```

Listing A.1: Befehle zur Reproduktion

B Instrumentierte Beispielfunktion

Listing B.1 zeigt die Funktion aus Listing 5.1 nach der Instrumentierung (Abschnitt 5.3), wieder in Quelltext zurückübersetzt. Die Sonden `_F[0]` bis `_F[7]` entsprechen den acht Zweigen aus Tabelle 5.1; die `else`-Blöcke der inneren Bedingung und der `while`-Schleife wurden ergänzt.

```
def f(a, b):
    r = 0
    t1 = b - a
    if b != -8:
        _F[0] = 1
        t2 = a - b
        if a < -39 or t1 != -13:
            _F[2] = 1
            t3 = t2 * 3 - 7
        else:
            _F[3] = 1
            t4 = b * 3 - 2
            w1 = a
            while w1 > -31:
                _F[4] = 1
                t5 = a + t4
                w1 = w1 - 17
            else:
                _F[5] = 1
            return t4
    else:
        _F[1] = 1
        r = r - a
    if b > a or a < -31:
        _F[6] = 1
        t6 = a + t1
    else:
        _F[7] = 1
        t7 = t1 * -2 - 8
    return r
```

Listing B.1: Instrumentierte Fassung von Listing 5.1